



# Deep Decision Making

EMEN 7104 Guest Lecture

Steven Morad

University of Macau

|                                 |    |
|---------------------------------|----|
| Decision Making .....           | 2  |
| The Agent and Environment ..... | 12 |
| Markov Decision Processes ..... | 17 |
| Agent Policies .....            | 36 |
| Policy Gradient .....           | 40 |

# Decision Making

---

# Decision Making

<https://www.youtube.com/watch?v=Zeyv1bN9v4A> GT

<https://www.youtube.com/watch?v=kopoLzvh5jY&t=1s> H&S

[https://www.youtube.com/watch?v=eHipy\\_j29Xw](https://www.youtube.com/watch?v=eHipy_j29Xw) DoTA

# Decision Making

Decision making applications include:

# Decision Making

Decision making applications include:

- Autonomous vehicles

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models
- Behavior modeling in psychology/ecology/biology

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models
- Behavior modeling in psychology/ecology/biology
- Finance

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models
- Behavior modeling in psychology/ecology/biology
- Finance
- Artificial General Intelligence?

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models
- Behavior modeling in psychology/ecology/biology
- Finance
- Artificial General Intelligence?
- Anywhere with cause and effect
  - Where you **change** the world by **interacting** with it

# Decision Making

Decision making applications include:

- Autonomous vehicles
- Video game NPCs
- Large language models
- Behavior modeling in psychology/ecology/biology
- Finance
- Artificial General Intelligence?
- Anywhere with cause and effect
  - Where you **change** the world by **interacting** with it
  - Standard ML: Detect traffic light in image
  - Decision Making: Detected a red traffic light, and I must stop or else

# Decision Making

**Lecture Goal:** Provide a proper understanding of the theoretical foundations of decision making

# Decision Making

~~**Lecture Goal:** Provide a proper understanding of the theoretical foundations of reinforcement learning~~

# Decision Making

~~**Lecture Goal:** Provide a proper understanding of the theoretical foundations of reinforcement learning~~

**Lecture Goal:** Give you enough information to learn decision making on your own

# Decision Making

What methods to we have for decision making?

# Decision Making

What methods to we have for decision making?

- Expert systems

# Decision Making

What methods to we have for decision making?

- Expert systems
- Fuzzy logic

# Decision Making

What methods to we have for decision making?

- Expert systems
- Fuzzy logic
- ...

# Decision Making

What methods to we have for decision making?

- ~~Expert systems~~
- ~~Fuzzy logic~~
- ...

# Decision Making

What methods to we have for decision making?

- ~~Expert systems~~
- ~~Fuzzy logic~~
- ...
- Reinforcement learning (RL)

# Decision Making

What methods to we have for decision making?

- ~~Expert systems~~
- ~~Fuzzy logic~~
- ...
- Reinforcement learning (RL)
- Imitation learning (IL)

# Decision Making

What methods to we have for decision making?

- ~~Expert systems~~
- ~~Fuzzy logic~~
- ...
- Reinforcement learning (RL)
- Imitation learning (IL)

RL and IL are the most popular decision making methods today

# Decision Making

What methods to we have for decision making?

- ~~Expert systems~~
- ~~Fuzzy logic~~
- ...
- Reinforcement learning (RL)
- Imitation learning (IL)

RL and IL are the most popular decision making methods today

- Work well with deep neural networks

# Decision Making



IL learns from a human expert

# Decision Making



IL learns from a human expert

- Easy to learn

# Decision Making



IL learns from a human expert

- Easy to learn
  - Learns **how**, no need to learn **why**

# Decision Making



IL learns from a human expert

- Easy to learn
  - Learns **how**, no need to learn **why**
- Only as good as the human expert

# Decision Making



RL learns without humans

# Decision Making



RL learns without humans

- Difficult to learn

# Decision Making



RL learns without humans

- Difficult to learn
  - Learns **how** and **why**

# Decision Making



RL learns without humans

- Difficult to learn
  - Learns **how** and **why**
- Can exceed human experts

# Decision Making

Both IL and RL use the same decision making framework

# Decision Making

Both IL and RL use the same decision making framework

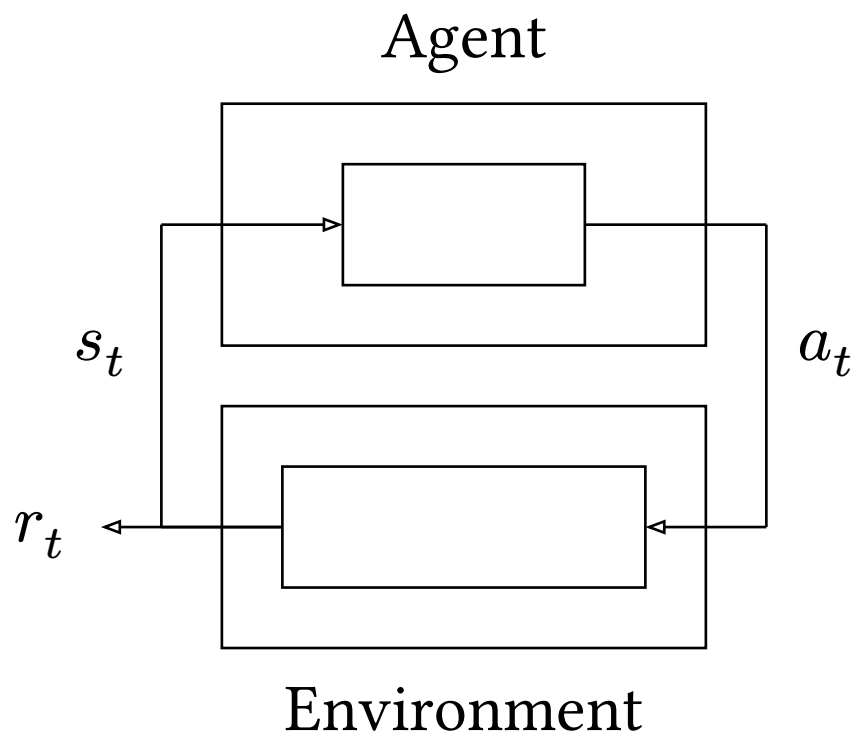
The only difference is the objective function

# The Agent and Environment

---

# The Agent and Environment

- In decision making, we have the **agent** and **environment**



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

# The Agent and Environment



Agent

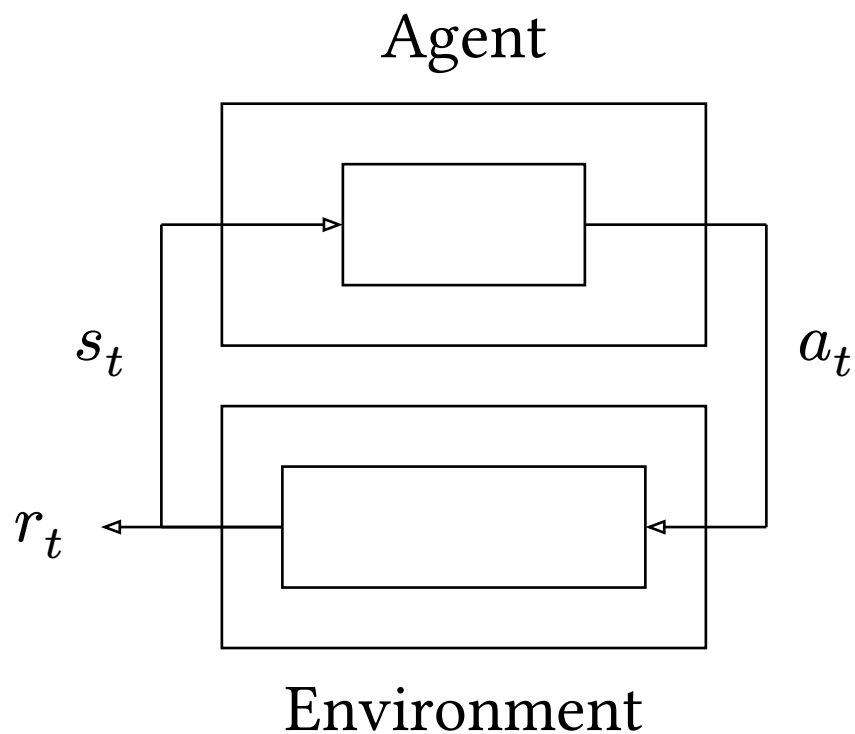


Environment

# The Agent and Environment

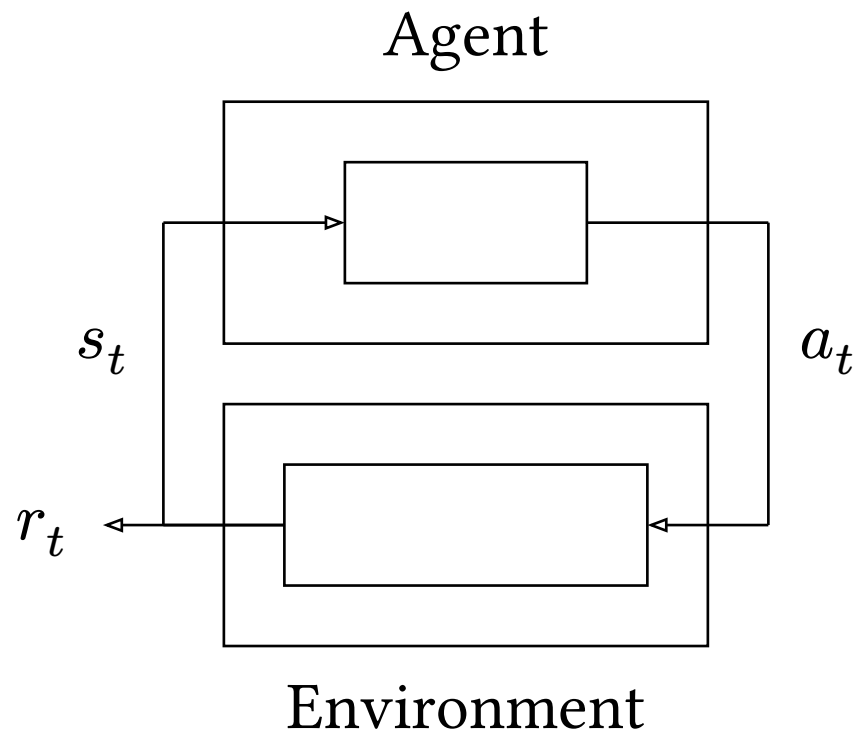
We have the **agent** and **environment**

The agent takes **actions** in the environment



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

# The Agent and Environment



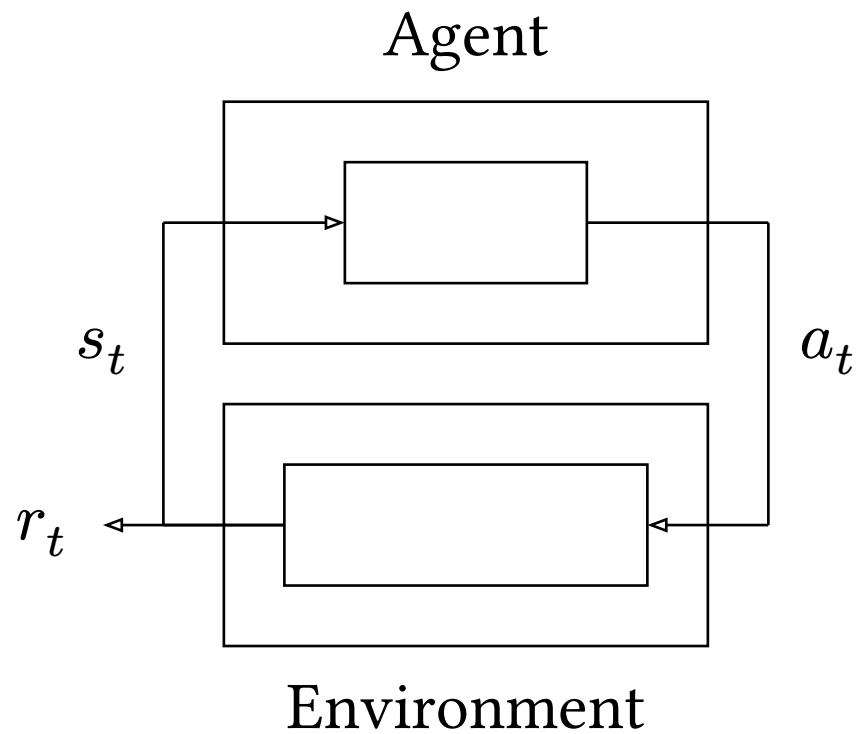
$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

We have the **agent** and **environment**

The agent takes **actions** in the environment

Actions change the environment **state**, producing an new state and **reward**

# The Agent and Environment



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

We have the **agent** and **environment**

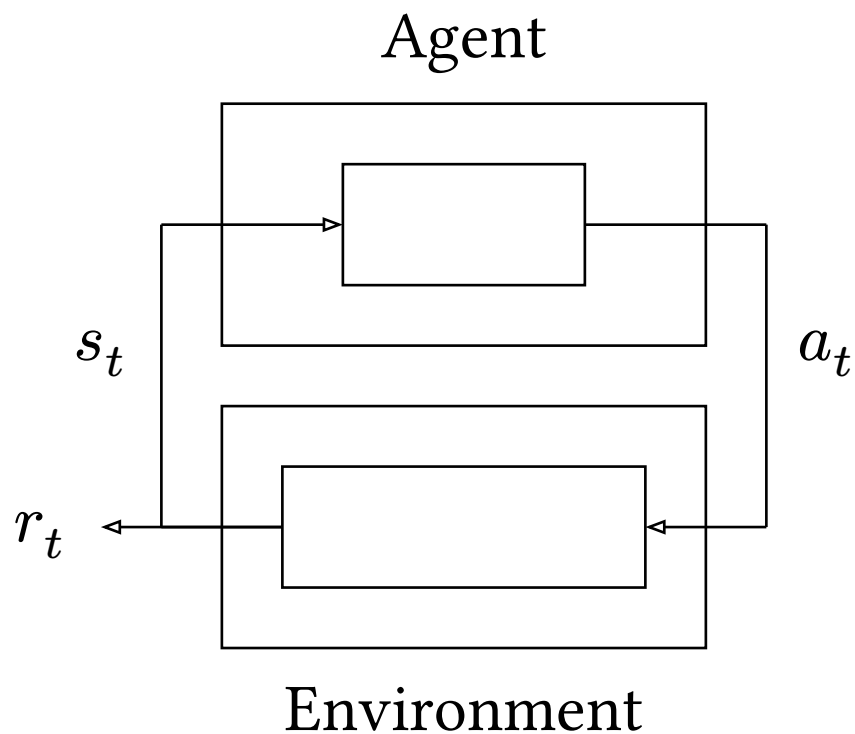
The agent takes **actions** in the environment

Actions change the environment **state**, producing an new state and **reward**

The cycle continues for  $t = 0, 1, \dots$

# The Agent and Environment

In RL, the agent maximizes the cumulative reward

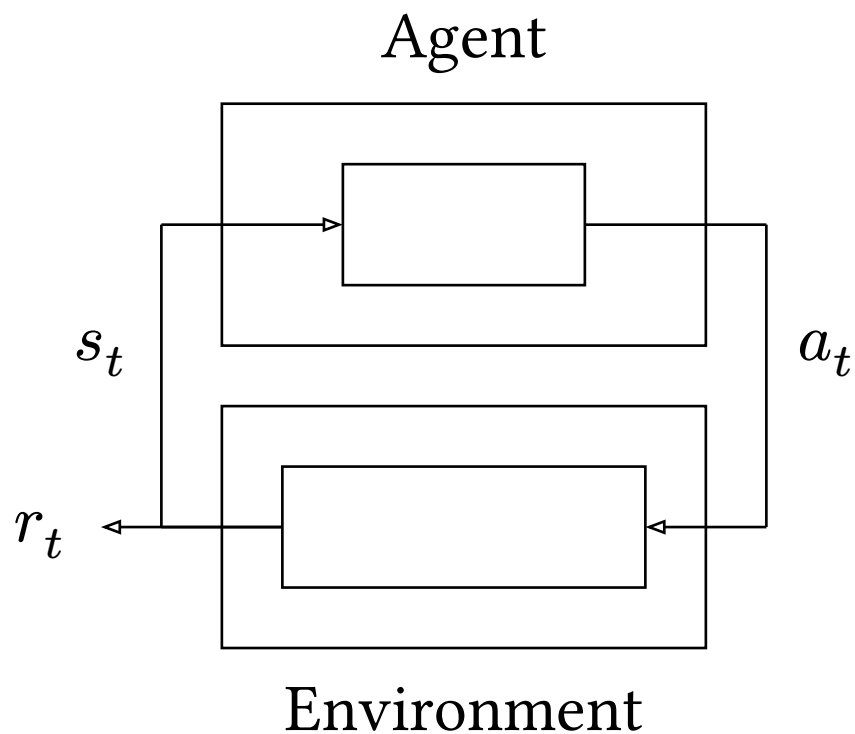


$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

# The Agent and Environment

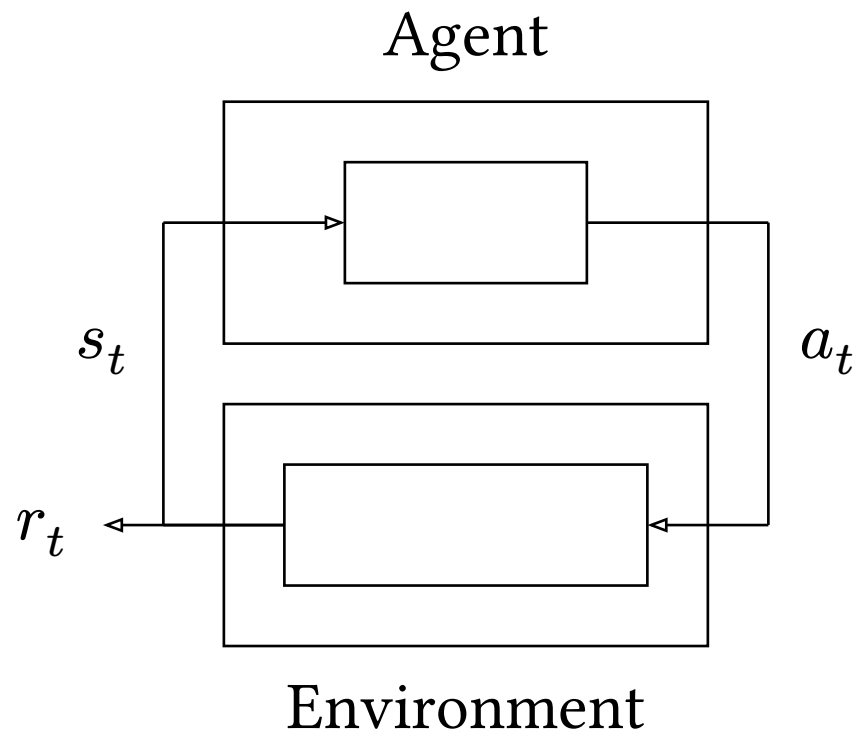
In RL, the agent maximizes the cumulative reward

$$r_1 + r_2 + \dots$$



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

# The Agent and Environment



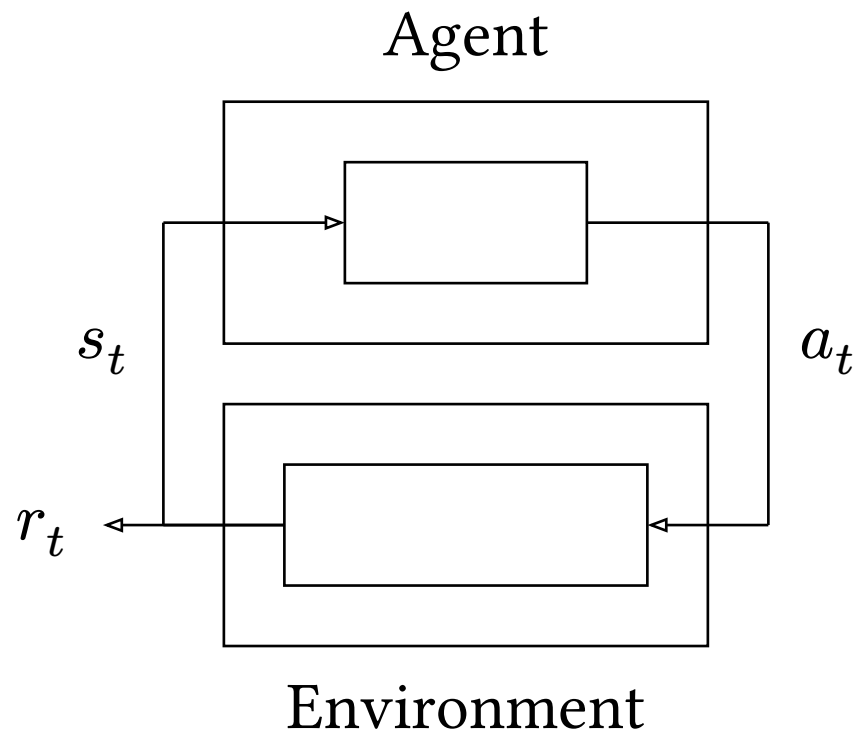
$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

In RL, the agent maximizes the cumulative reward

$$r_1 + r_2 + \dots$$

In IL, the agent copies a human expert

# The Agent and Environment



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

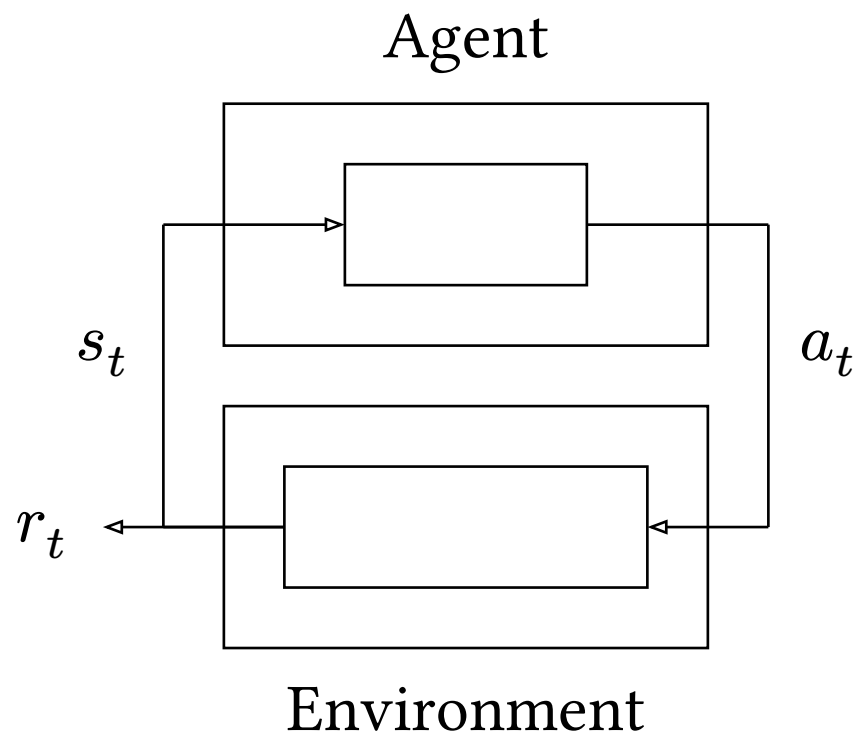
In RL, the agent maximizes the cumulative reward

$$r_1 + r_2 + \dots$$

In IL, the agent copies a human expert

$$a_1, a_2, \dots$$

# The Agent and Environment



$s_t$ : state,  $a_t$ : action,  $r_t$ : reward

In RL, the agent maximizes the cumulative reward

$$r_1 + r_2 + \dots$$

In IL, the agent copies a human expert

$$a_1, a_2, \dots$$

Let us formally define the environment

# Markov Decision Processes

---

# Markov Decision Processes

By definition, RL and IL solve **Markov Decision Processes (MDPs)**.

# Markov Decision Processes

By definition, RL and IL solve **Markov Decision Processes (MDPs)**.

To solve a problem using RL/IL, we first must convert the problem into an MDP. We call this converted problem the **environment**.

# Markov Decision Processes

By definition, RL and IL solve **Markov Decision Processes (MDPs)**.

To solve a problem using RL/IL, we first must convert the problem into an MDP. We call this converted problem the **environment**.

How you structure your problem is **critical** – more important than which algorithms you use, how much compute you have, etc.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

$\gamma \in [0, 1)$  is the **discount factor**

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

$\gamma \in [0, 1)$  is the **discount factor**

Let us briefly explain these terms.

# Markov Decision Processes

$S$  is the set of states known as the **state space**.

# Markov Decision Processes

$S$  is the set of states known as the **state space**.

Recall that the environment is always changing

# Markov Decision Processes

$S$  is the set of states known as the **state space**.

Recall that the environment is always changing

We need a way to describe what state the environment is in

# Markov Decision Processes

$S$  is the set of states known as the **state space**.

Recall that the environment is always changing

We need a way to describe what state the environment is in

If the environment is a table, the state space might describe the positions of all objects on the table  $[x_1 \ y_1 \ x_2 \ y_2 \ \dots]$

# Markov Decision Processes

$A$  is the set of actions known as the **action space**

# Markov Decision Processes

$A$  is the set of actions known as the **action space**

What capabilities does the agent have?

# Markov Decision Processes

$A$  is the set of actions known as the **action space**

What capabilities does the agent have?

For the table example, I can apply a force to a specific object on the table

$$[F_x \ F_y \ i]$$

# Markov Decision Processes

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

# Markov Decision Processes

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

Sometimes called “transition dynamics”, “state transition matrix”, etc

# Markov Decision Processes

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

Sometimes called “transition dynamics”, “state transition matrix”, etc

“Rules” of the environment, determines the (stochastic) evolution of the environment

# Markov Decision Processes

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

Sometimes called “transition dynamics”, “state transition matrix”, etc

“Rules” of the environment, determines the (stochastic) evolution of the environment

$$T \left( \underbrace{\begin{bmatrix} x & y & \varphi \end{bmatrix}}_{\text{state}}, \underbrace{\begin{bmatrix} F_x & F_y & \tau \end{bmatrix}}_{\text{action}} \right) = \underbrace{\Delta \begin{bmatrix} x & y & \varphi \end{bmatrix}}_{\text{new state}}$$

# Markov Decision Processes

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

Sometimes called “transition dynamics”, “state transition matrix”, etc

“Rules” of the environment, determines the (stochastic) evolution of the environment

$$T \left( \underbrace{\begin{bmatrix} x & y & \varphi \end{bmatrix}}_{\text{state}}, \underbrace{\begin{bmatrix} F_x & F_y & \tau \end{bmatrix}}_{\text{action}} \right) = \underbrace{\Delta \begin{bmatrix} x & y & \varphi \end{bmatrix}}_{\text{new state}}$$

This is a **Markov** decision process because transition dynamics are **conditionally independent** of past states and actions

$$T(s_t, a_t \mid s_{t-1}, a_{t-1}, \dots, s_0, a_0) = T(s_t, a_t)$$

# Markov Decision Processes

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

# Markov Decision Processes

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

Produces reward based on the state

# Markov Decision Processes

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

Produces reward based on the state

Reward function determines agent behavior

# Markov Decision Processes

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

Produces reward based on the state

Reward function determines agent behavior

+1 for if object touches the floor, or +1 if object reaches the centre

# Markov Decision Processes

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

Produces reward based on the state

Reward function determines agent behavior

+1 for if object touches the floor, or +1 if object reaches the centre

Only used in RL, IL ignores rewards

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

$R : S \rightarrow \mathbb{R}$  is the **reward function**.

# Markov Decision Processes

**Definition:** An MDP is a 5-tuple  $(S, A, T, R, \gamma)$

$S$  is the set of states known as the **state space**.

$A$  is the set of actions known as the **action space**

$T : S \times A \rightarrow \Delta S$  The **state transition function**.

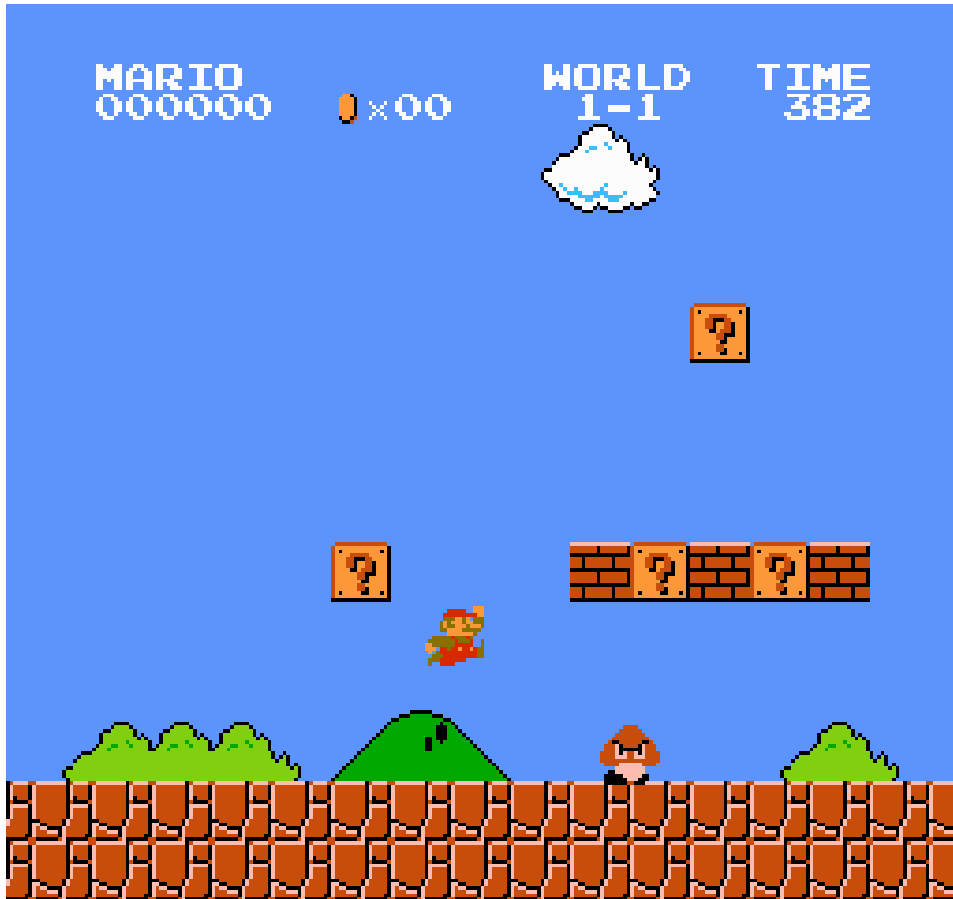
$R : S \rightarrow \mathbb{R}$  is the **reward function**.

$\gamma \in [0, 1)$  is the **discount factor**

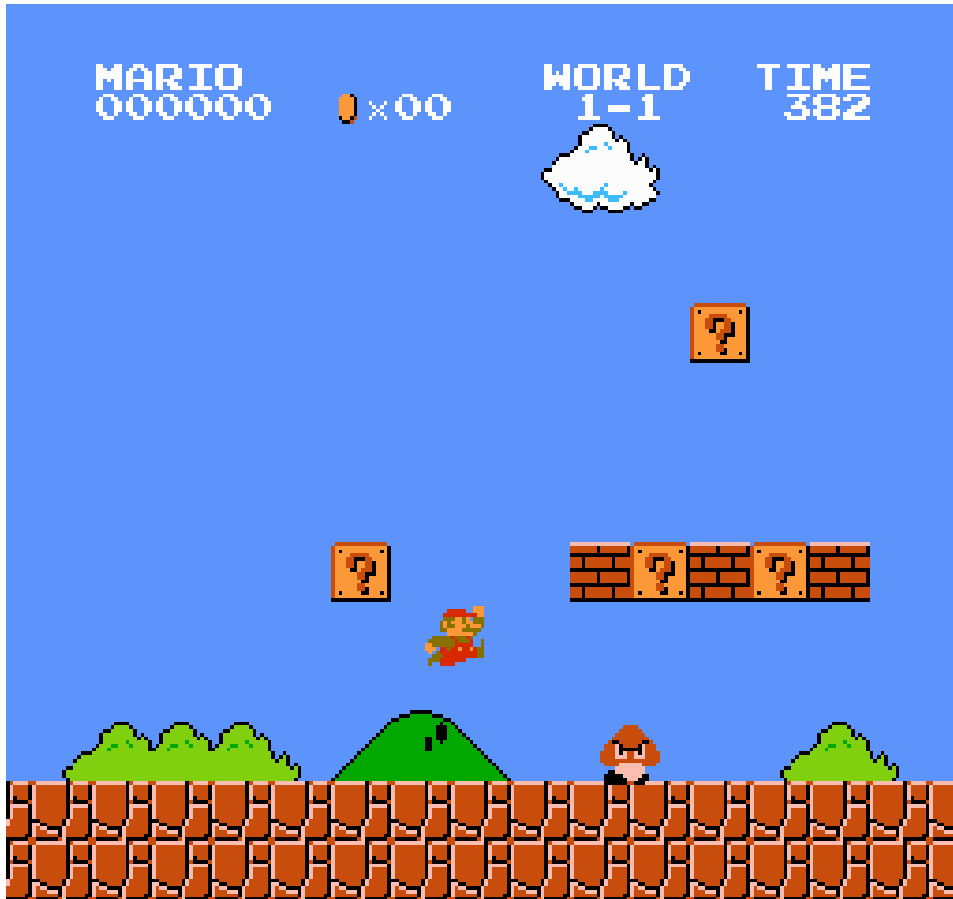
**Let us create an example MDP**

# Markov Decision Processes

Super Mario Bros. is a video game about Mario, an Italian plumber



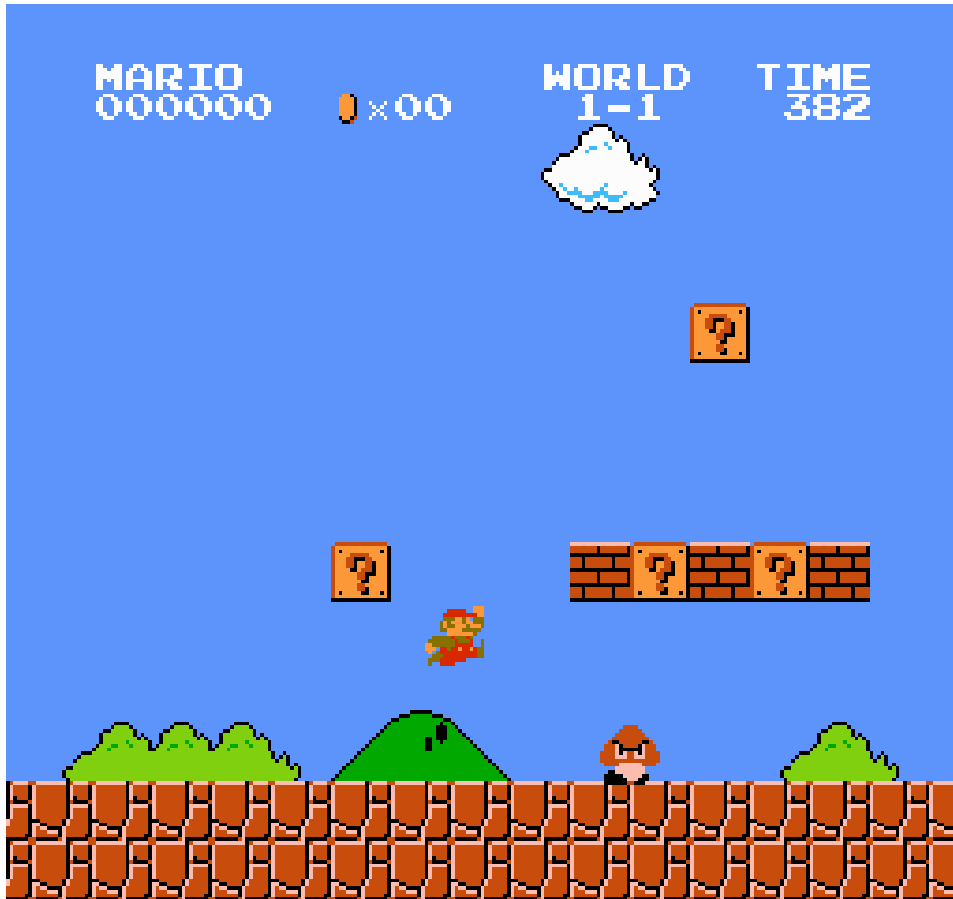
# Markov Decision Processes



Super Mario Bros. is a video game about Mario, an Italian plumber

Mario can move and jump

# Markov Decision Processes

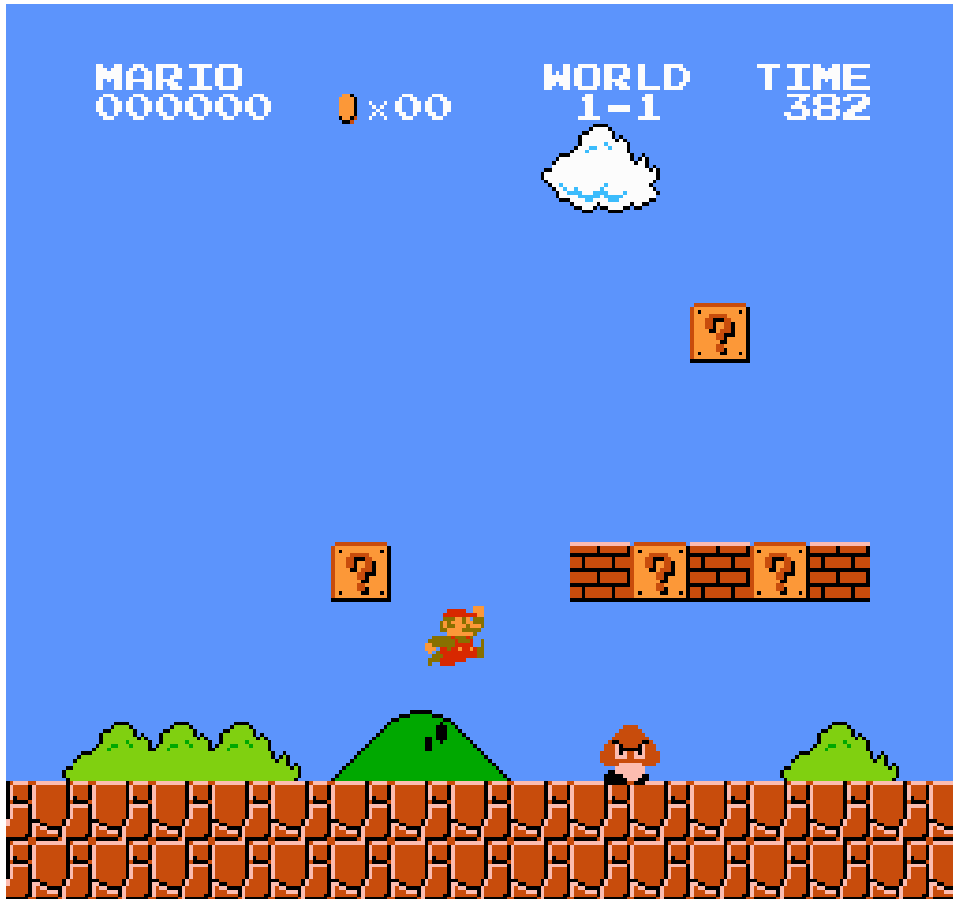


Super Mario Bros. is a video game about Mario, an Italian plumber

Mario can move and jump

Other units, such as the Goombas, walk around. Touching one kills you unless you land on their head, killing them instead

# Markov Decision Processes



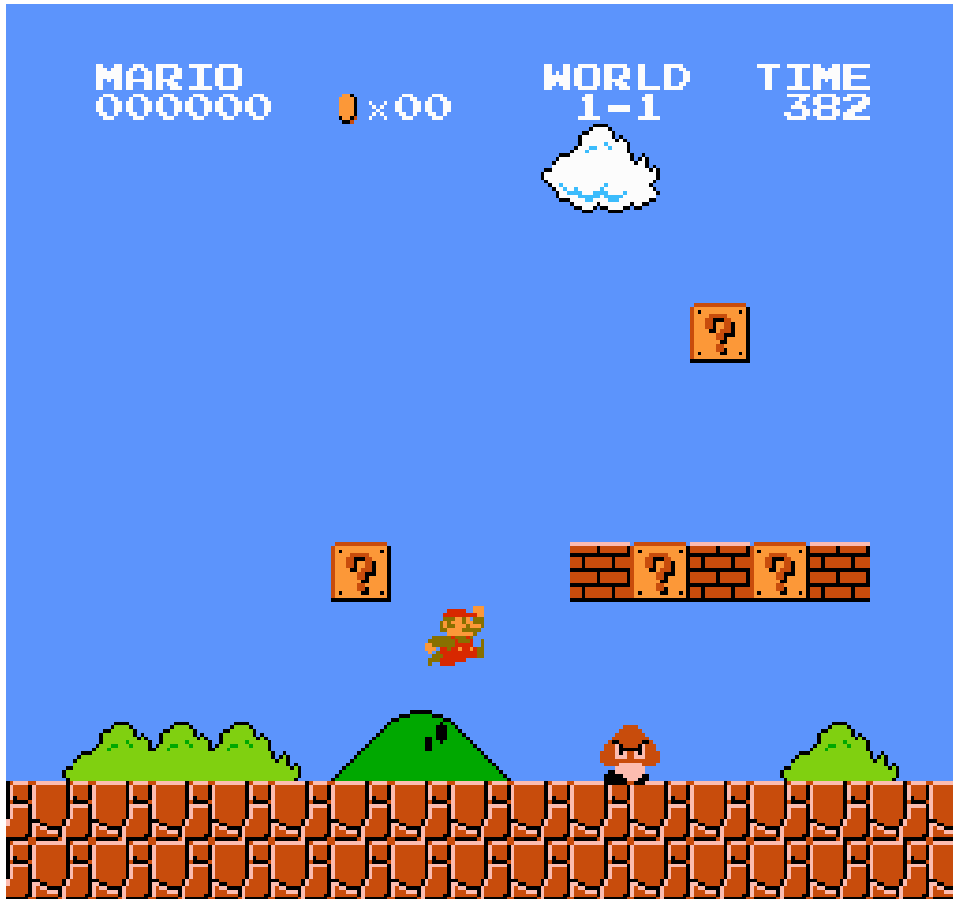
Super Mario Bros. is a video game about Mario, an Italian plumber

Mario can move and jump

Other units, such as the Goombas, walk around. Touching one kills you unless you land on their head, killing them instead

Bumping question mark blocks gives you items like mushrooms

# Markov Decision Processes



Super Mario Bros. is a video game about Mario, an Italian plumber

Mario can move and jump

Other units, such as the Goombas, walk around. Touching one kills you unless you land on their head, killing them instead

Bumping question mark blocks gives you items like mushrooms

You collect coins and have a time limit and score

# Markov Decision Processes



Super Mario Bros. is a video game about Mario, an Italian plumber

Mario can move and jump

Other units, such as the Goombas, walk around. Touching one kills you unless you land on their head, killing them instead

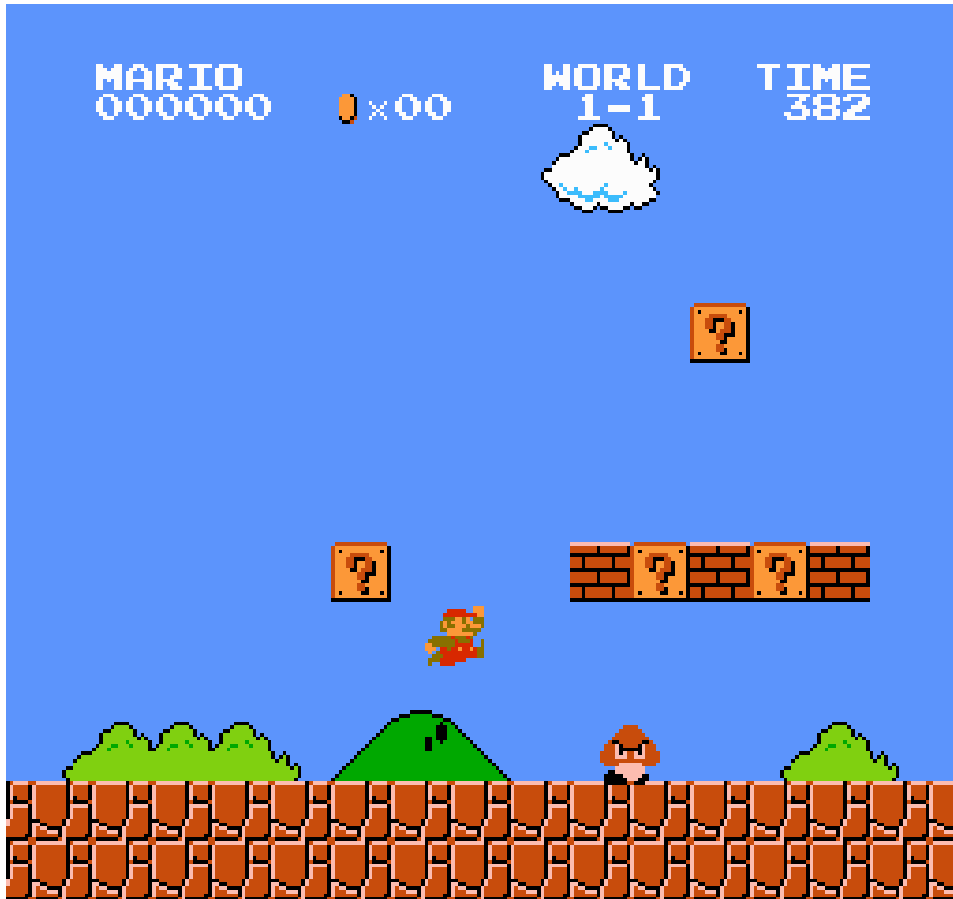
Bumping question mark blocks gives you items like mushrooms

You collect coins and have a time limit and score

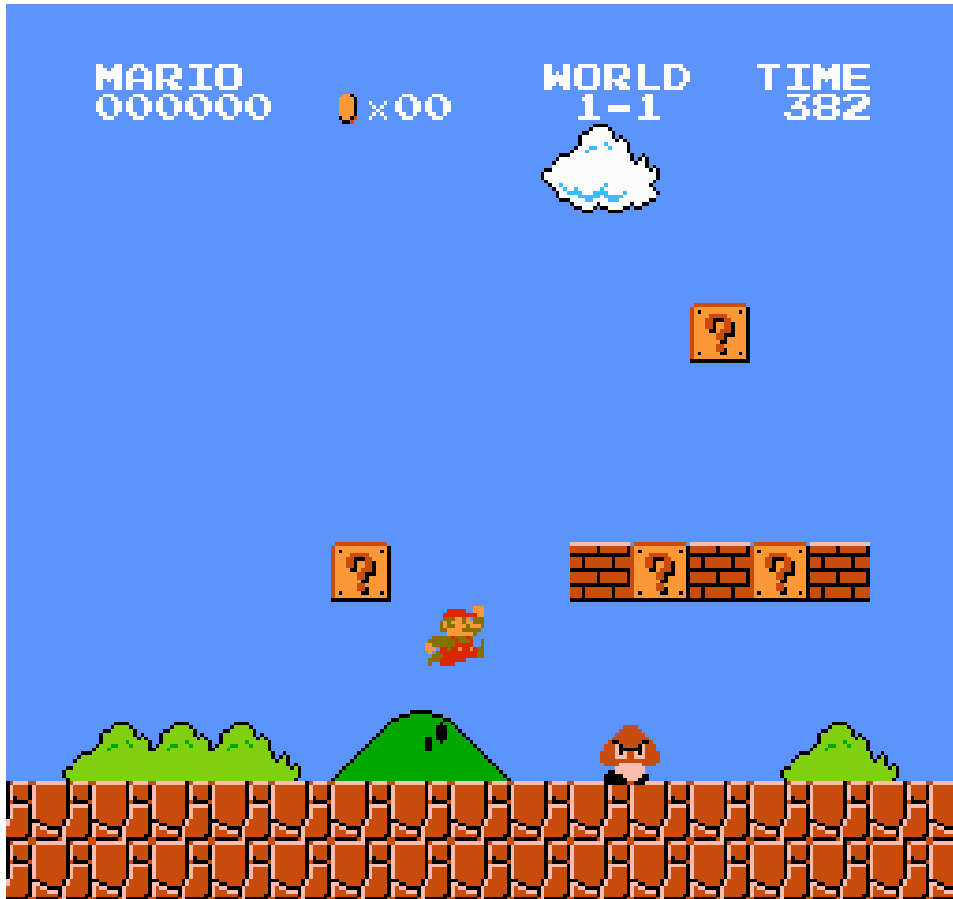
**Task:** Define Super Mario Bros. as an MDP

# Markov Decision Processes

State Space ( $S$ )?



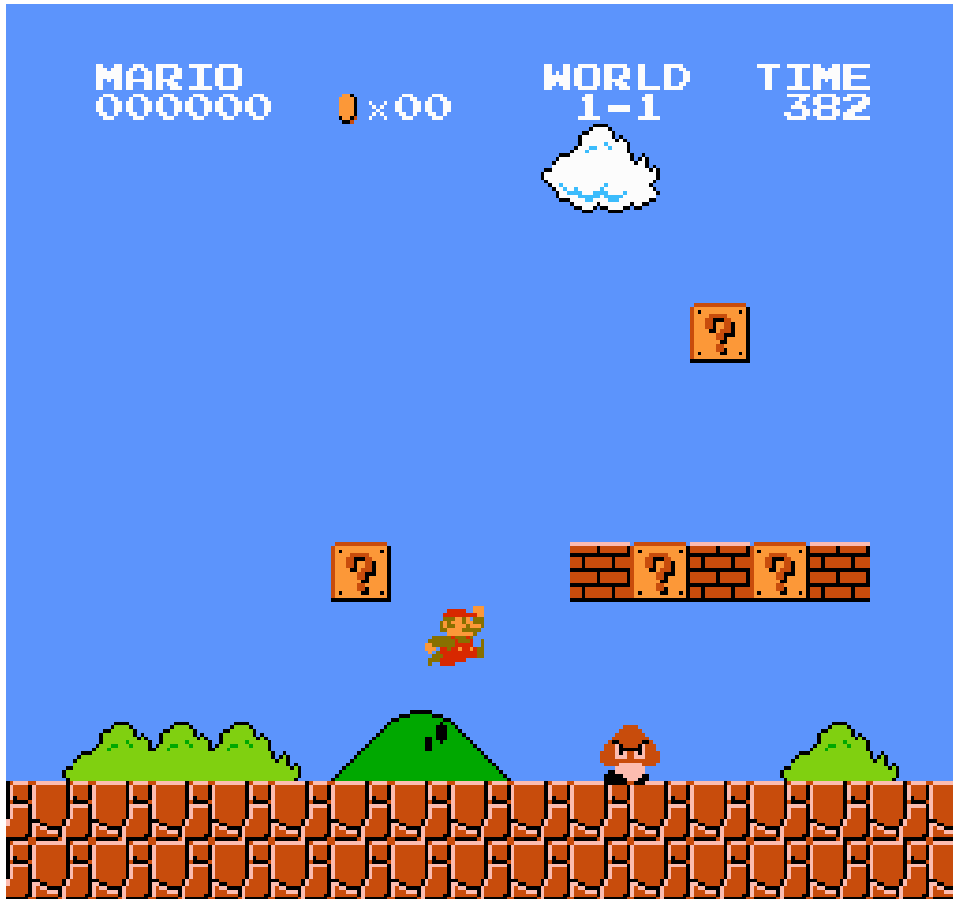
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario

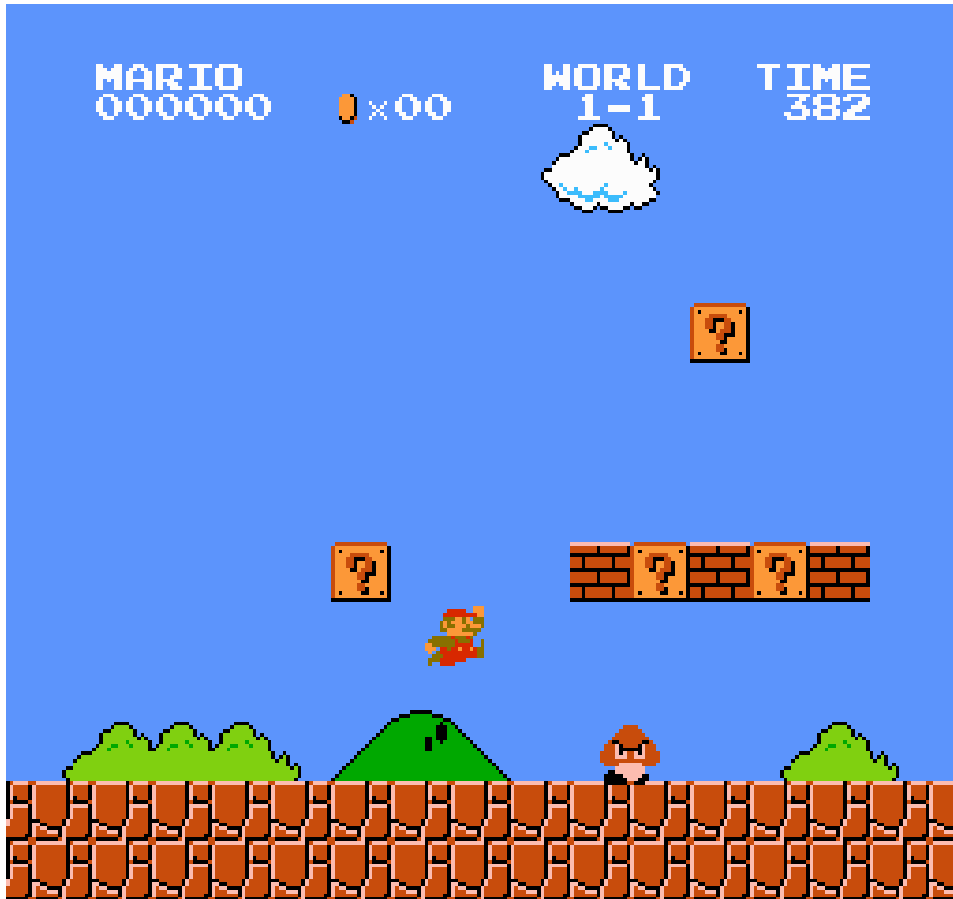
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score

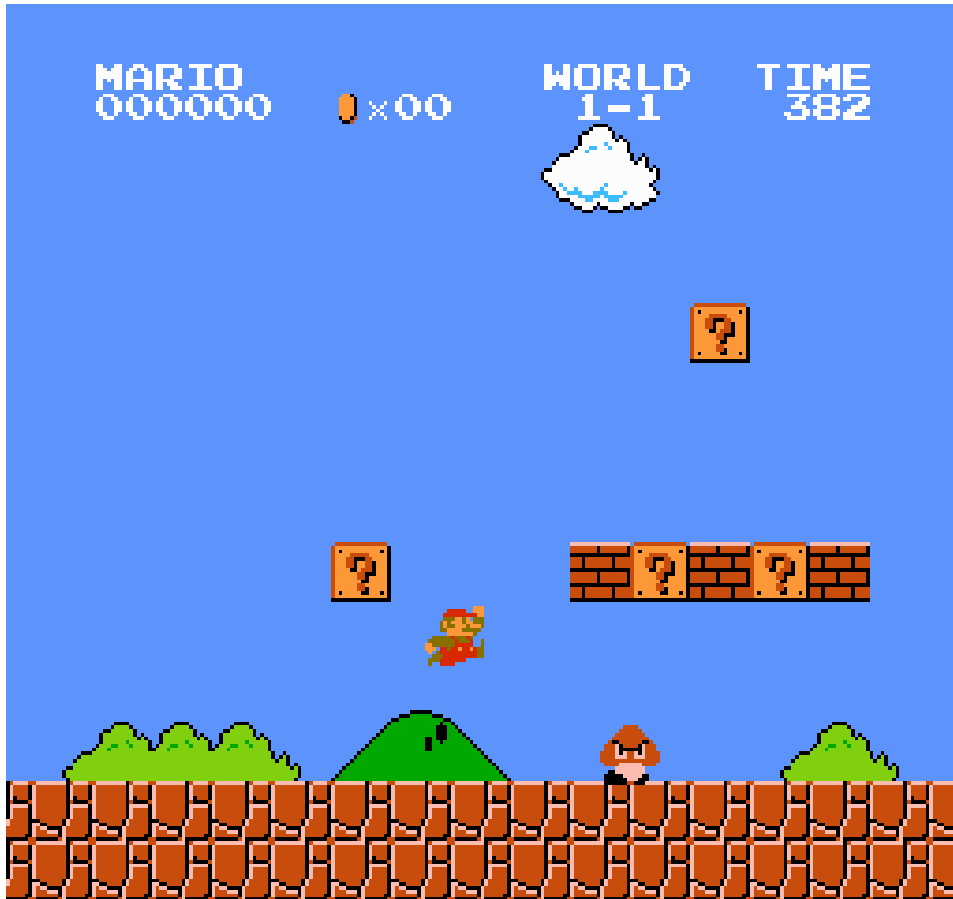
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score
- Number of coins collected

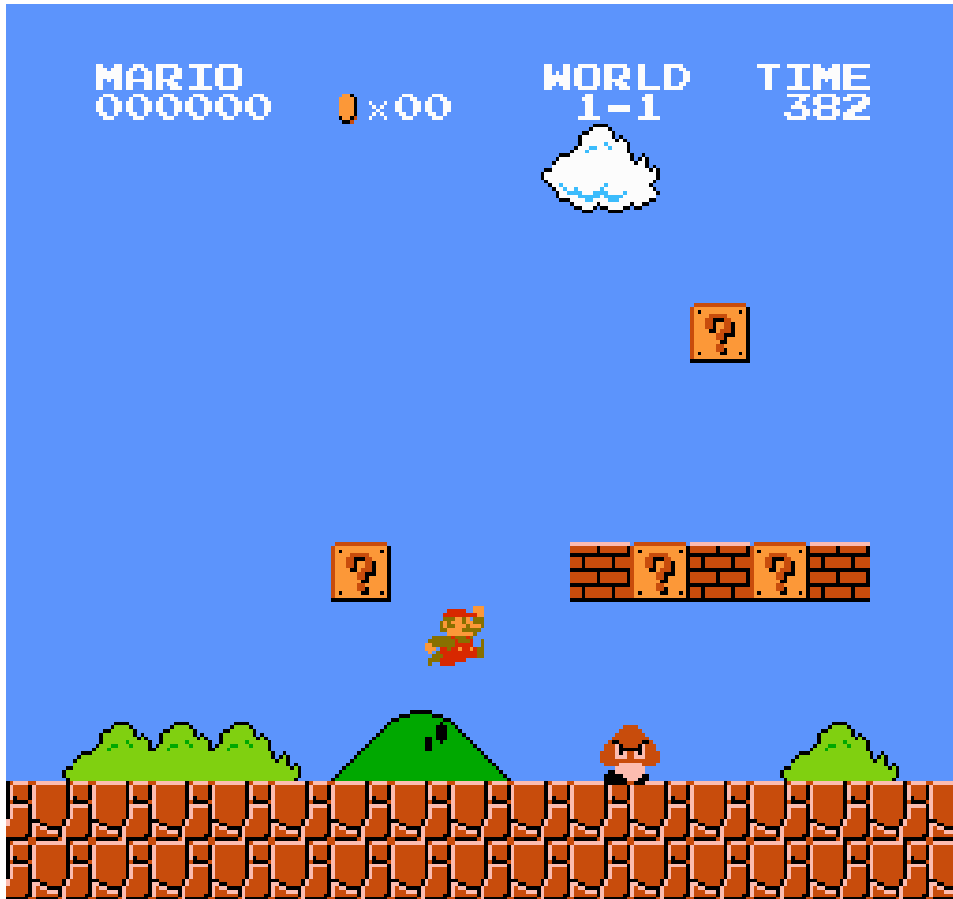
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score
- Number of coins collected
- The time remaining

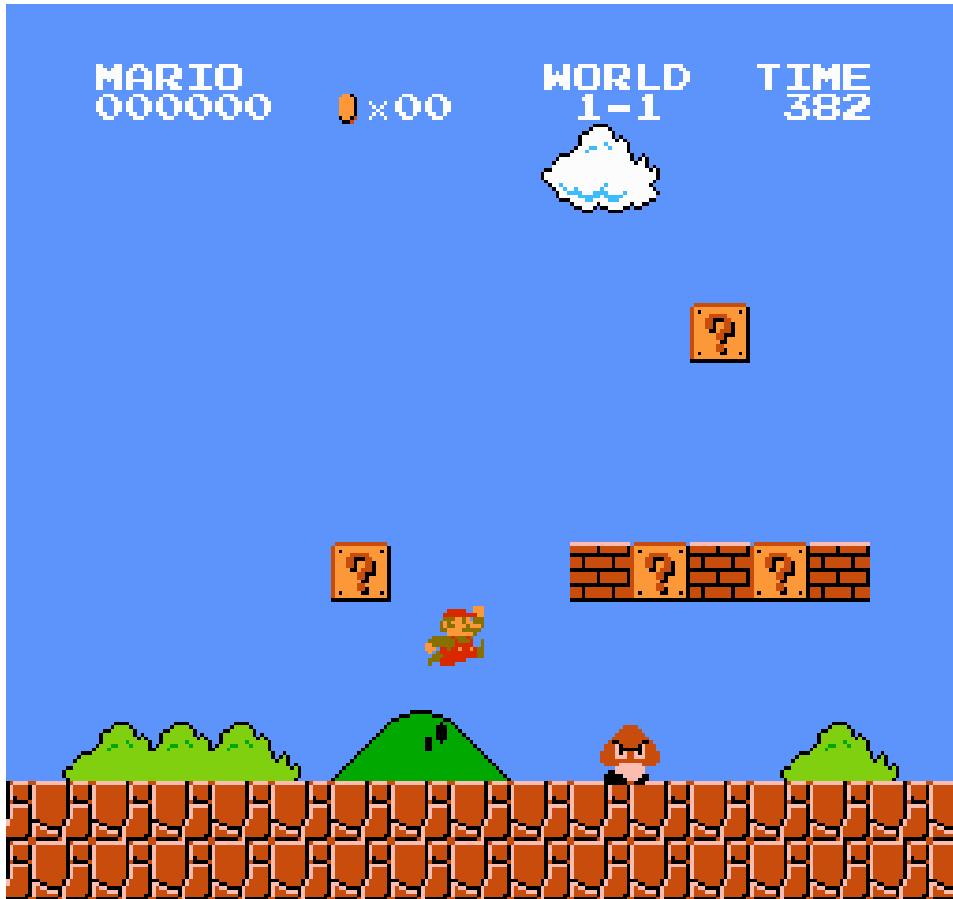
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score
- Number of coins collected
- The time remaining
- Which question blocks have been opened

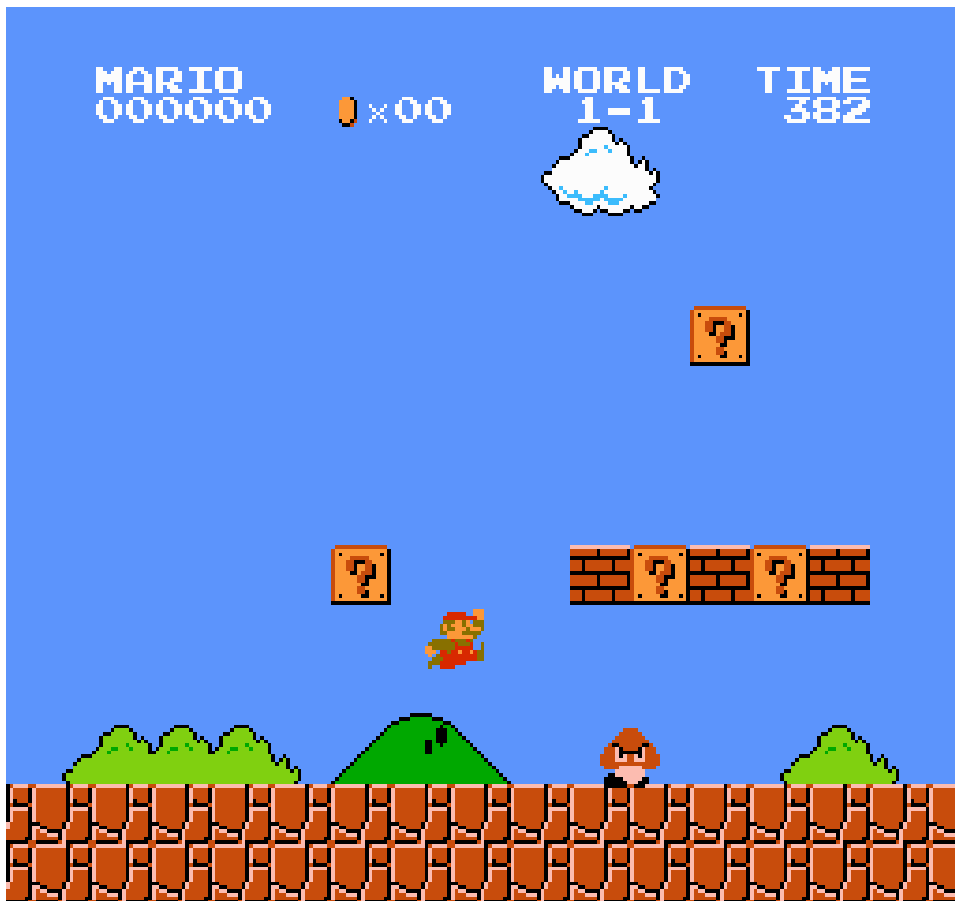
# Markov Decision Processes



## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score
- Number of coins collected
- The time remaining
- Which question blocks have been opened
- Goomba position/velocity and squished/not squished

# Markov Decision Processes



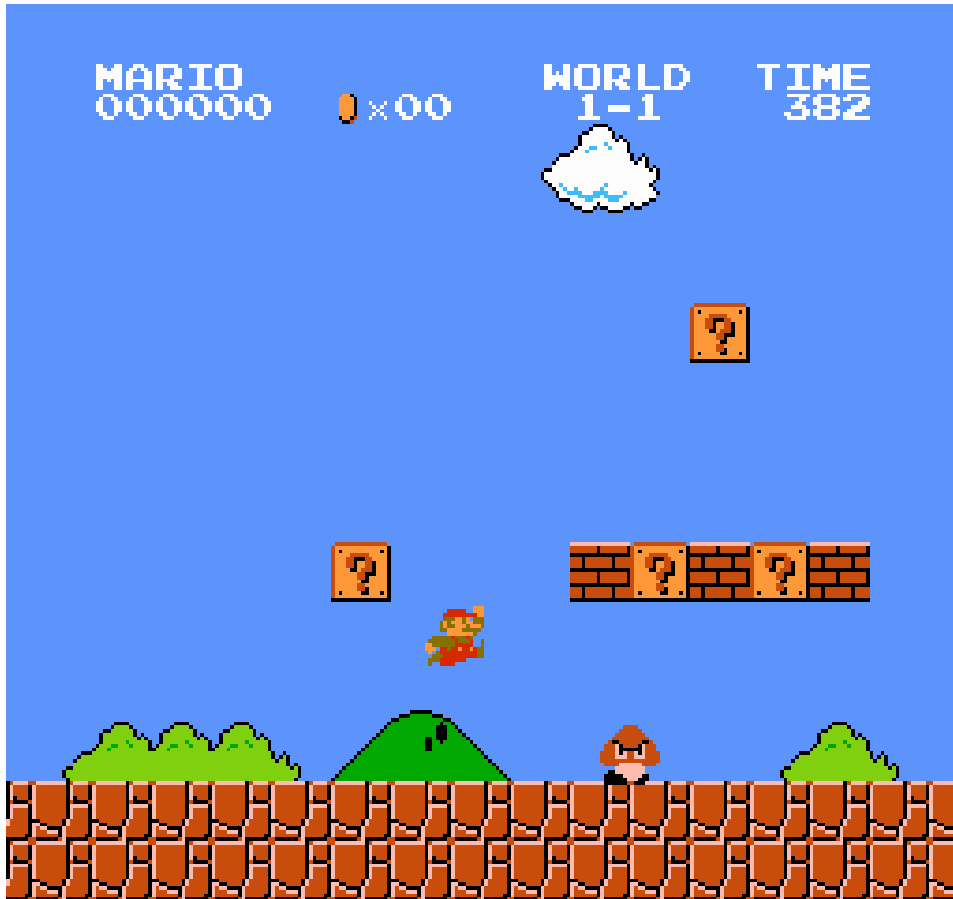
## State Space ( $S$ )?

- The position and velocity  $(x, y, \dot{x}, \dot{y})$  of Mario
- The score
- Number of coins collected
- The time remaining
- Which question blocks have been opened
- Goomba position/velocity and squished/not squished

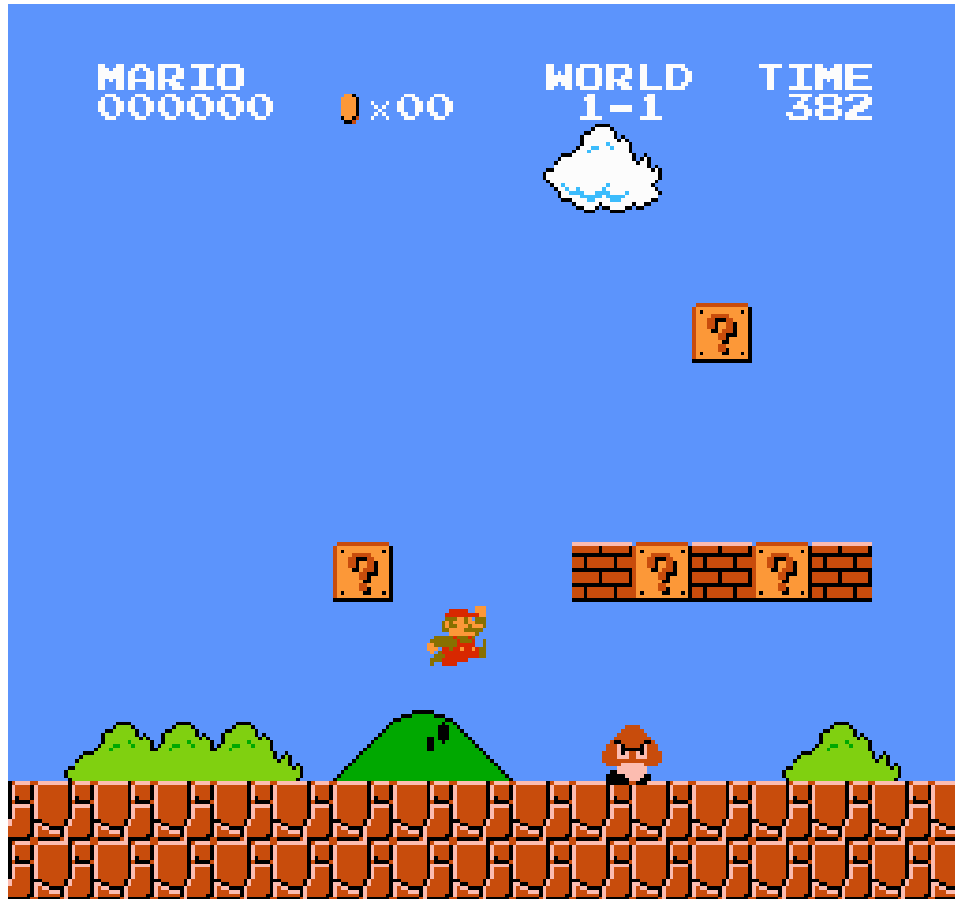
$$S = \{\mathbb{R}^4, \mathbb{Z}_+, \mathbb{Z}_+, \mathbb{Z}_+, \{0, 1\}^m, \mathbb{R}^{4 \times k}, \{0, 1\}^k\}$$

# Markov Decision Processes

Action Space ( $A$ )?



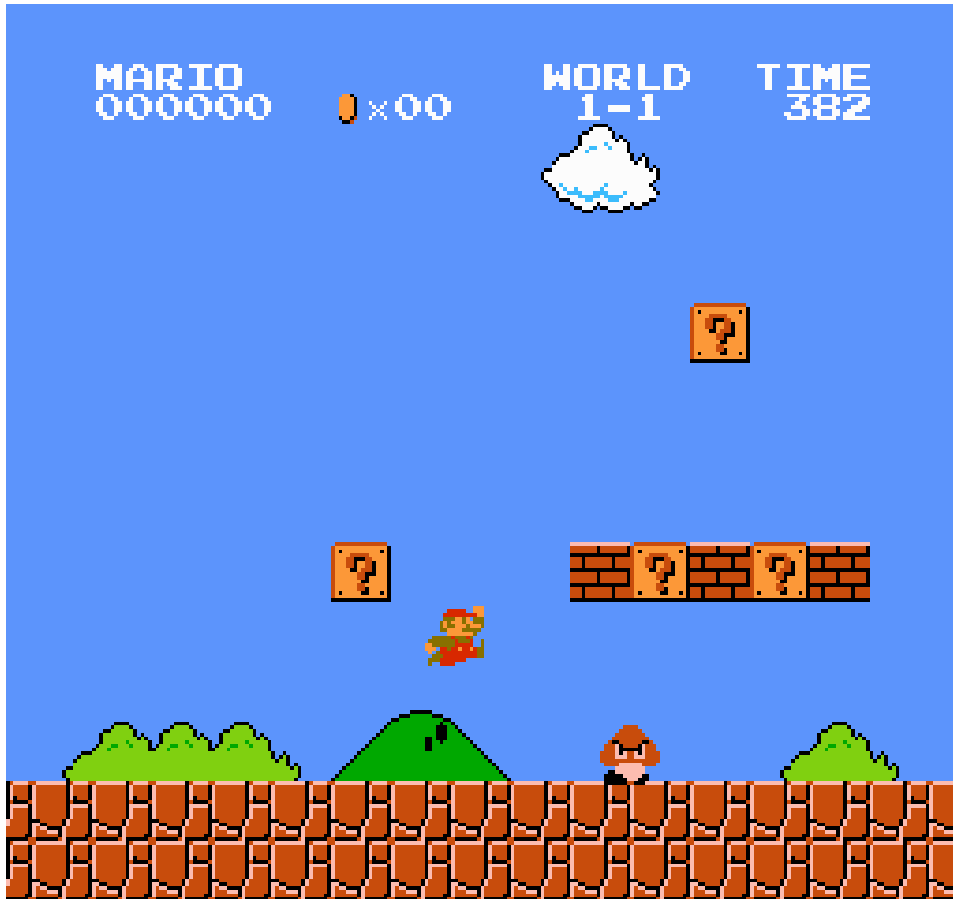
# Markov Decision Processes



## Action Space ( $A$ )?

- Acceleration of Mario  $\ddot{x}$

# Markov Decision Processes

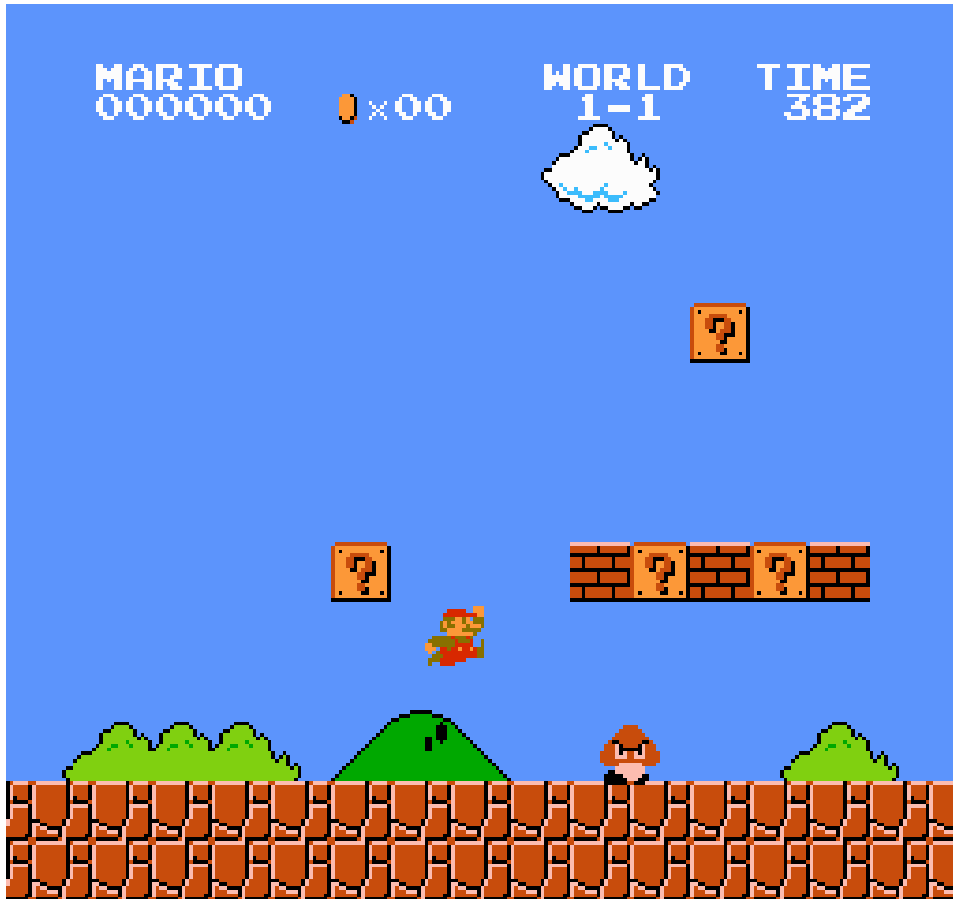


## Action Space ( $A$ )?

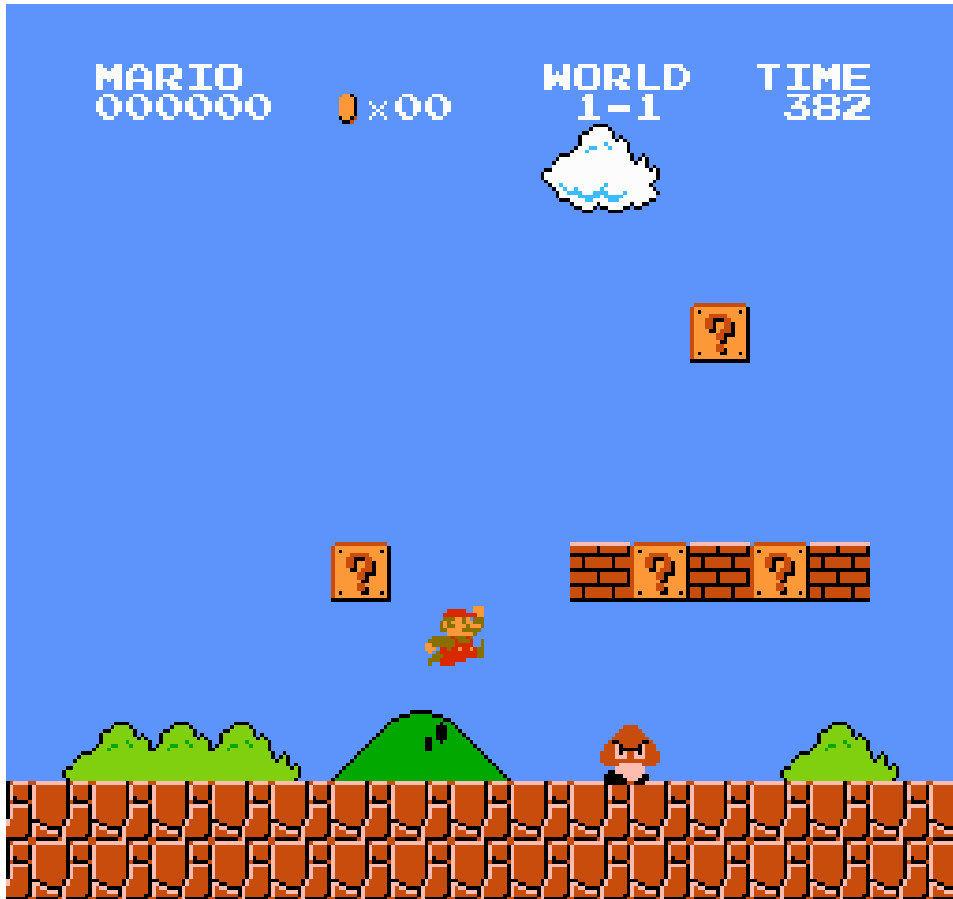
- Acceleration of Mario  $\ddot{x}$ 
  - But when playing Mario, we cannot explicitly set  $\ddot{x}$

# Markov Decision Processes

Action Space ( $A$ )?



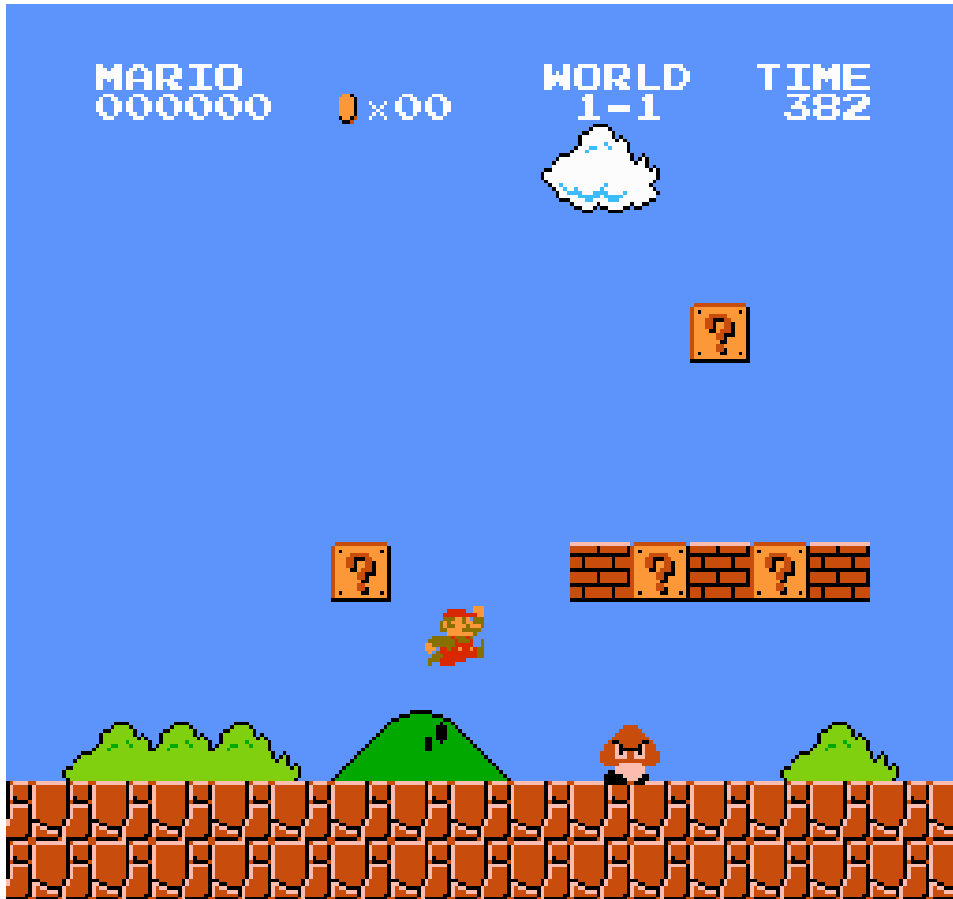
# Markov Decision Processes



## Action Space ( $A$ )?

- The Nintendo controller has  $A, B, \uparrow, \downarrow, \leftarrow, \rightarrow$  buttons

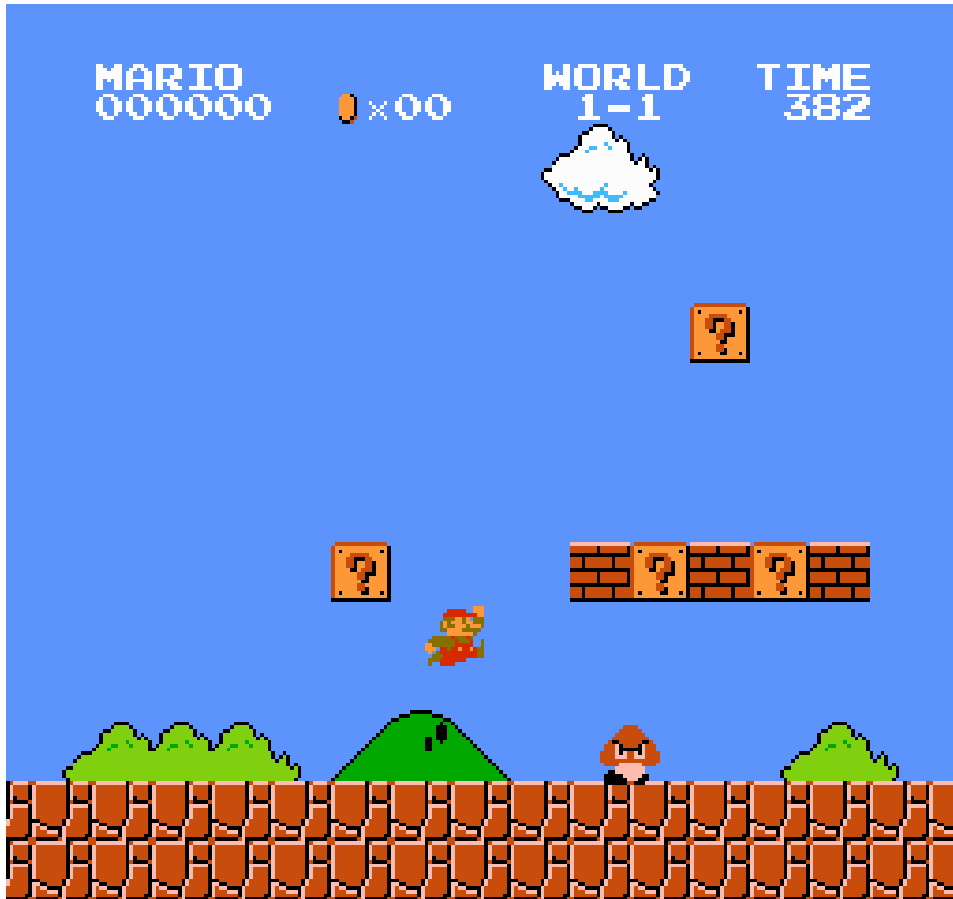
# Markov Decision Processes



## Action Space ( $A$ )?

- The Nintendo controller has  $A, B, \uparrow, \downarrow, \leftarrow, \rightarrow$  buttons
  - $A = \{A, B, \uparrow, \downarrow, \leftarrow, \rightarrow\}$

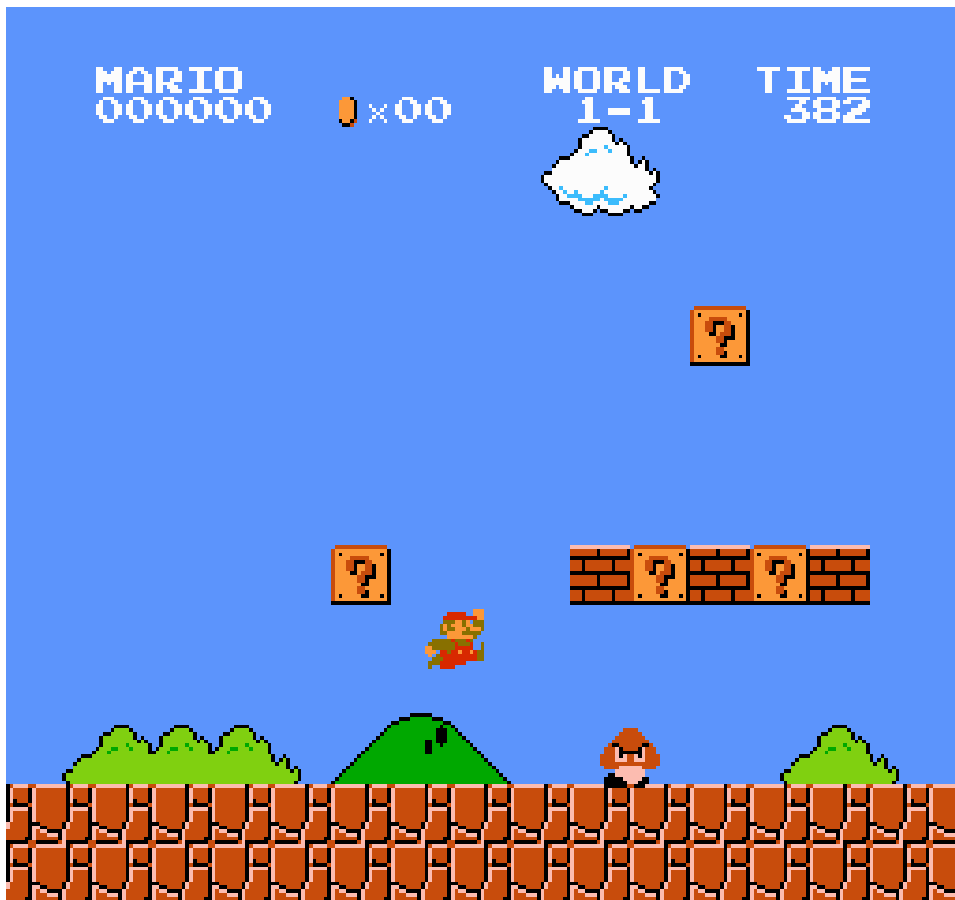
# Markov Decision Processes



## Action Space ( $A$ )?

- The Nintendo controller has  $A, B, \uparrow, \downarrow, \leftarrow, \rightarrow$  buttons
  - $A = \{A, B, \uparrow, \downarrow, \leftarrow, \rightarrow\}$ 
    - Cannot represent multiple buttons at once

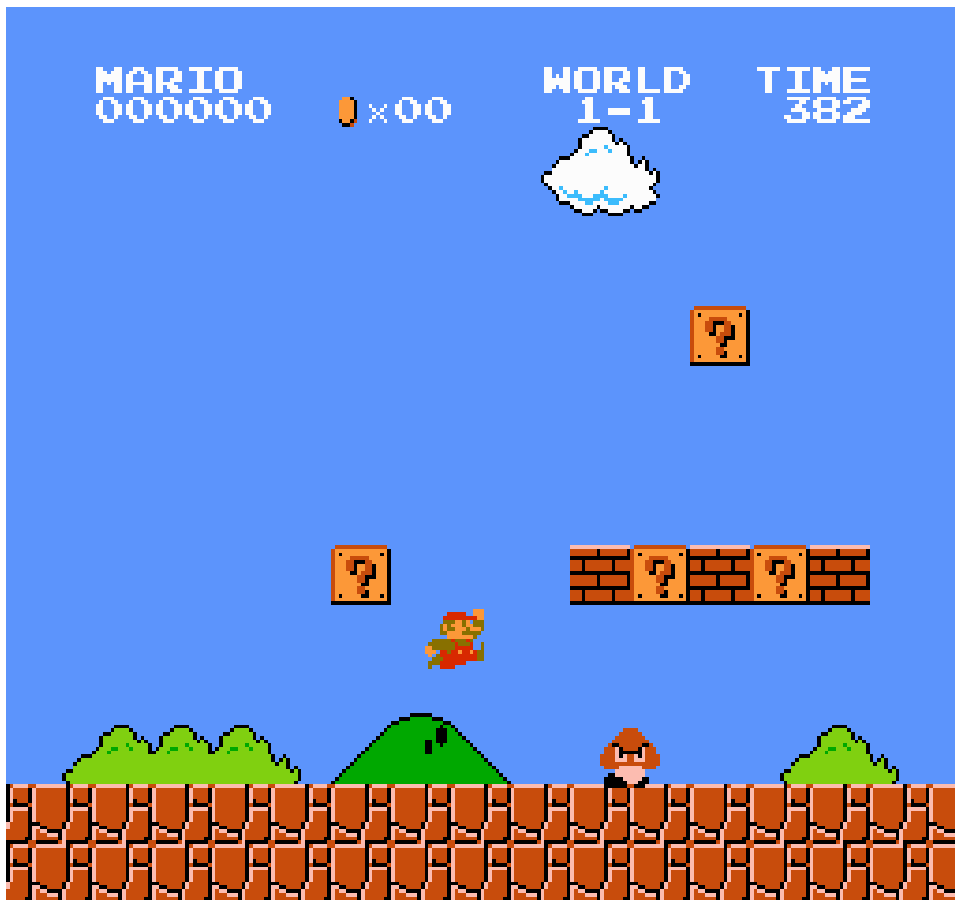
# Markov Decision Processes



## Action Space ( $A$ )?

- The Nintendo controller has  $A, B, \uparrow, \downarrow, \leftarrow, \rightarrow$  buttons
  - $A = \{A, B, \uparrow, \downarrow, \leftarrow, \rightarrow\}$ 
    - Cannot represent multiple buttons at once
  - $A = \{0, 1\}^6$

# Markov Decision Processes

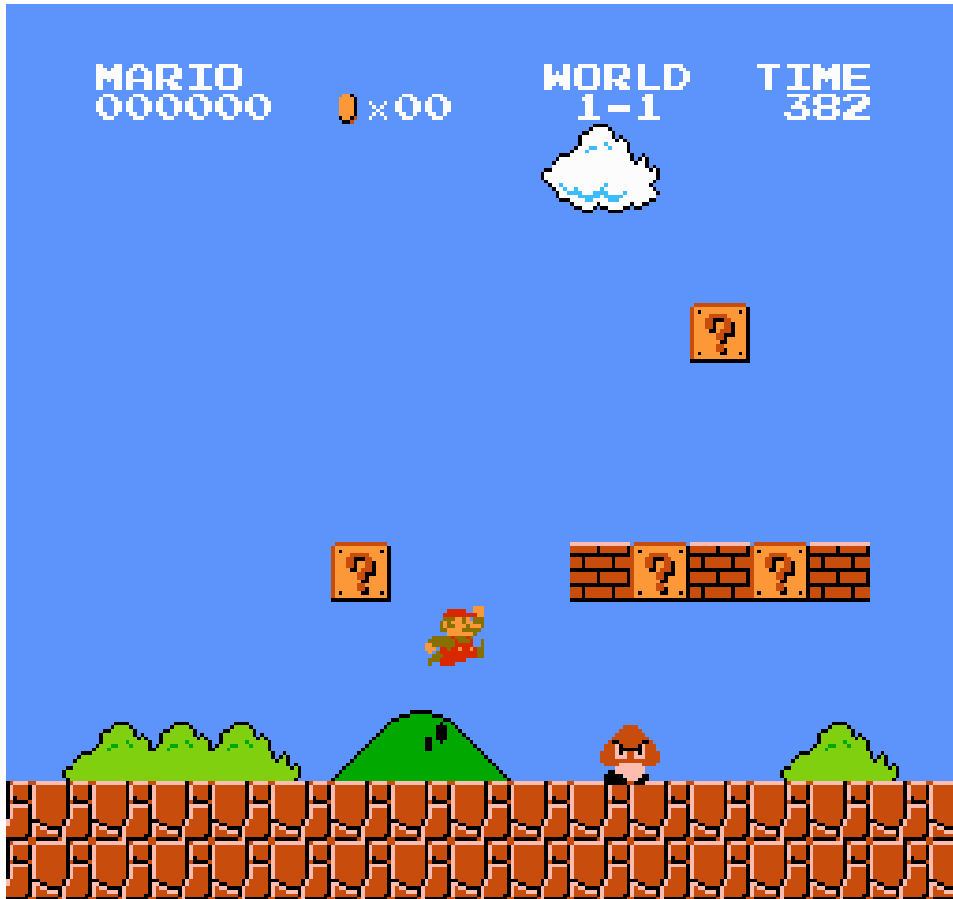


## Action Space ( $A$ )?

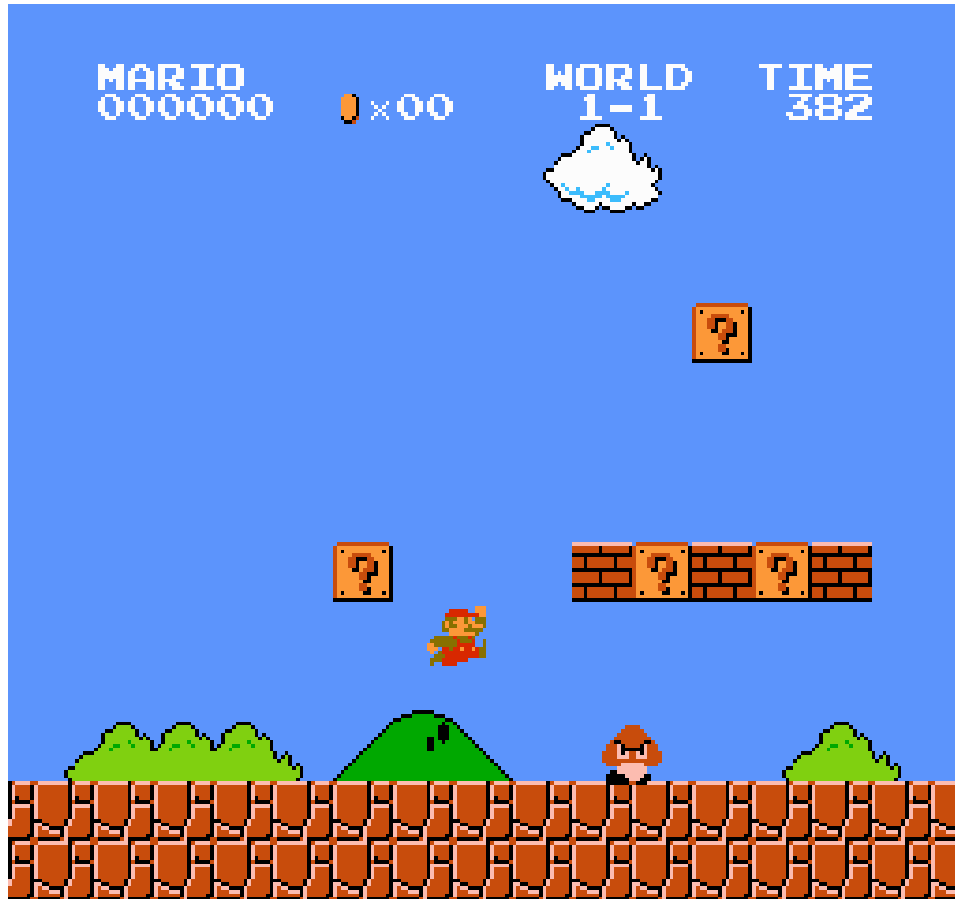
- The Nintendo controller has  $A, B, \uparrow, \downarrow, \leftarrow, \rightarrow$  buttons
  - $A = \{A, B, \uparrow, \downarrow, \leftarrow, \rightarrow\}$ 
    - Cannot represent multiple buttons at once
  - $A = \{0, 1\}^6$
  - $\left\{ \underbrace{\{0, 1, 2, 3, 4\}}_{\emptyset, \text{direction}} \times \underbrace{\{0, 1, 2, 3\}}_{\emptyset, a, b, a+b} \right\}$

# Markov Decision Processes

Transition Function ( $T$ )?



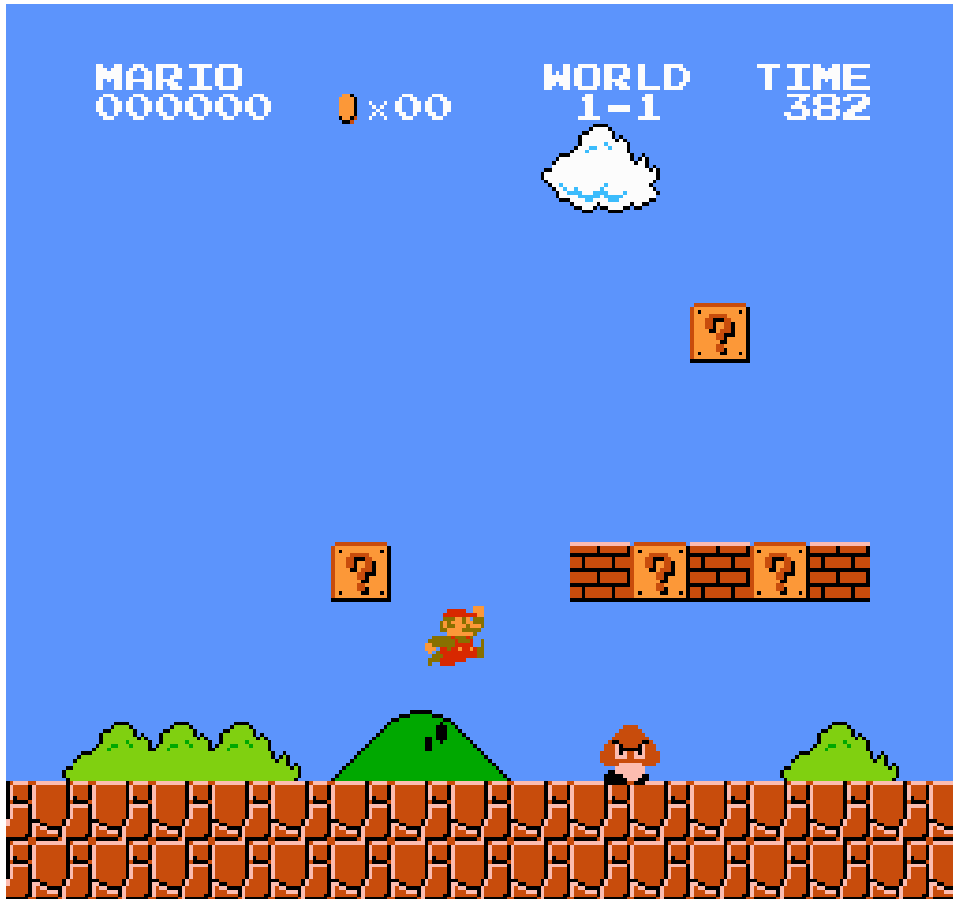
# Markov Decision Processes



Transition Function ( $T$ )?

- $T([x \ \dot{x} \ \dots], \rightarrow) = ?$

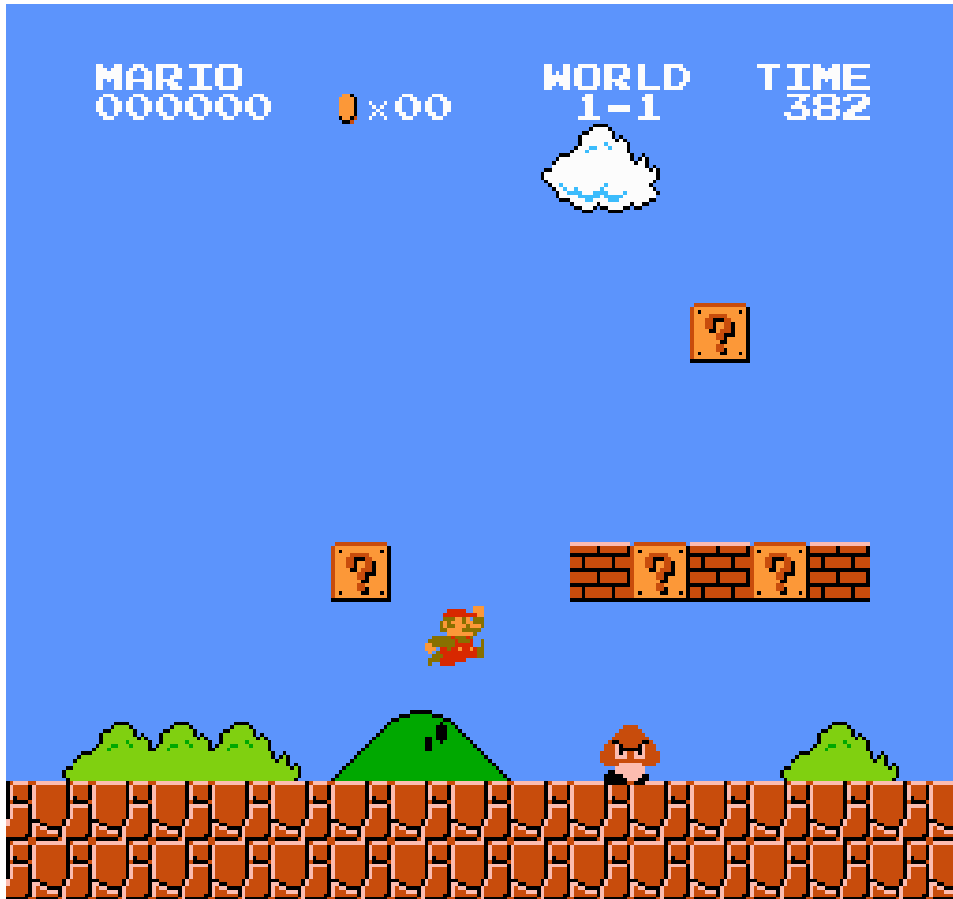
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy

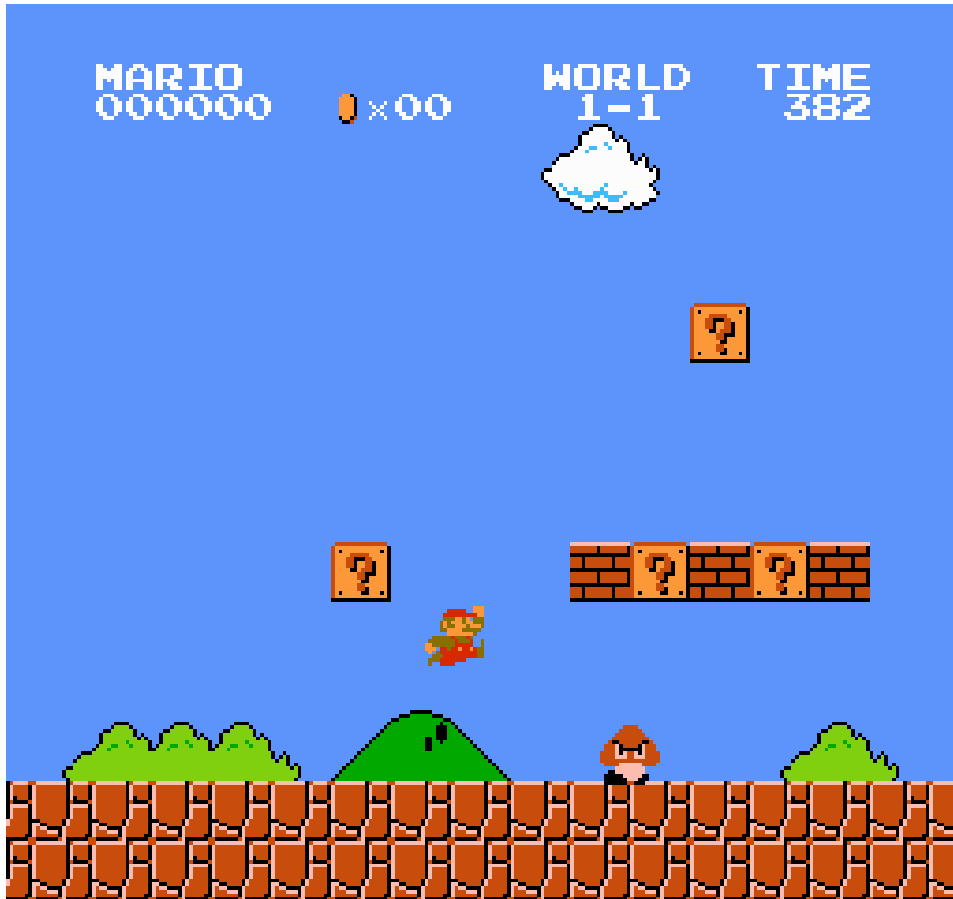
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ \dots], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$

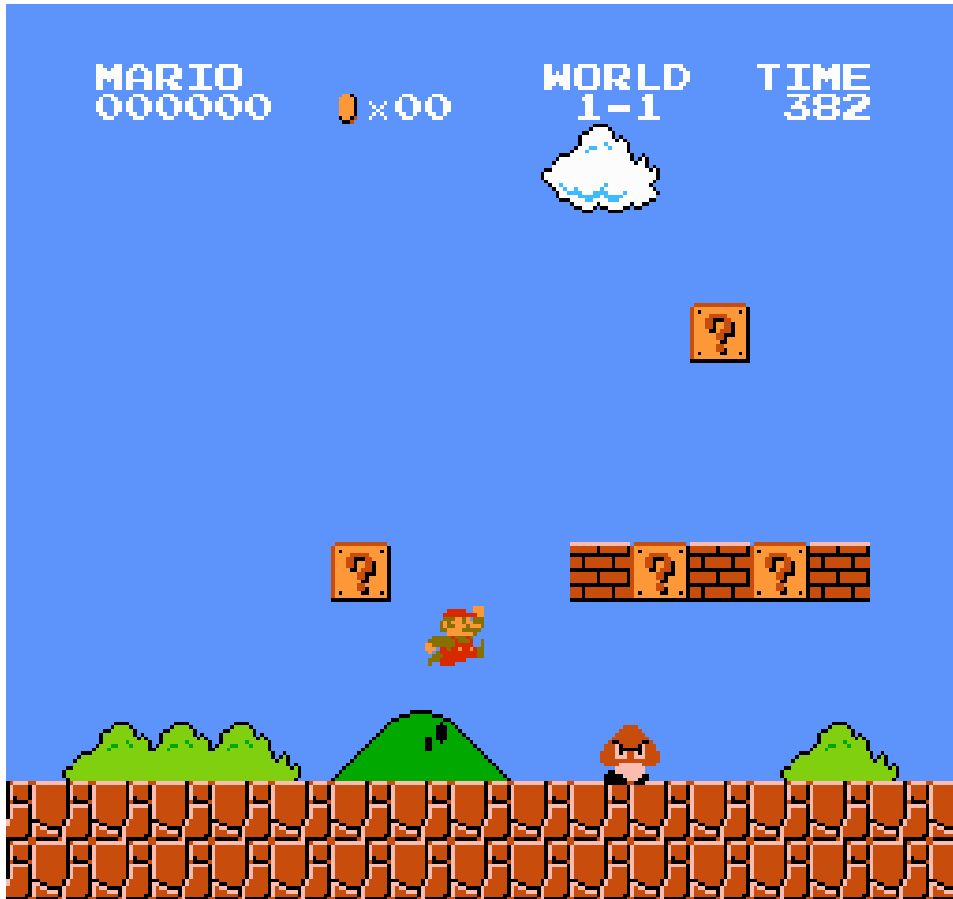
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$
    - $\dot{x} = [0 \ 0]$

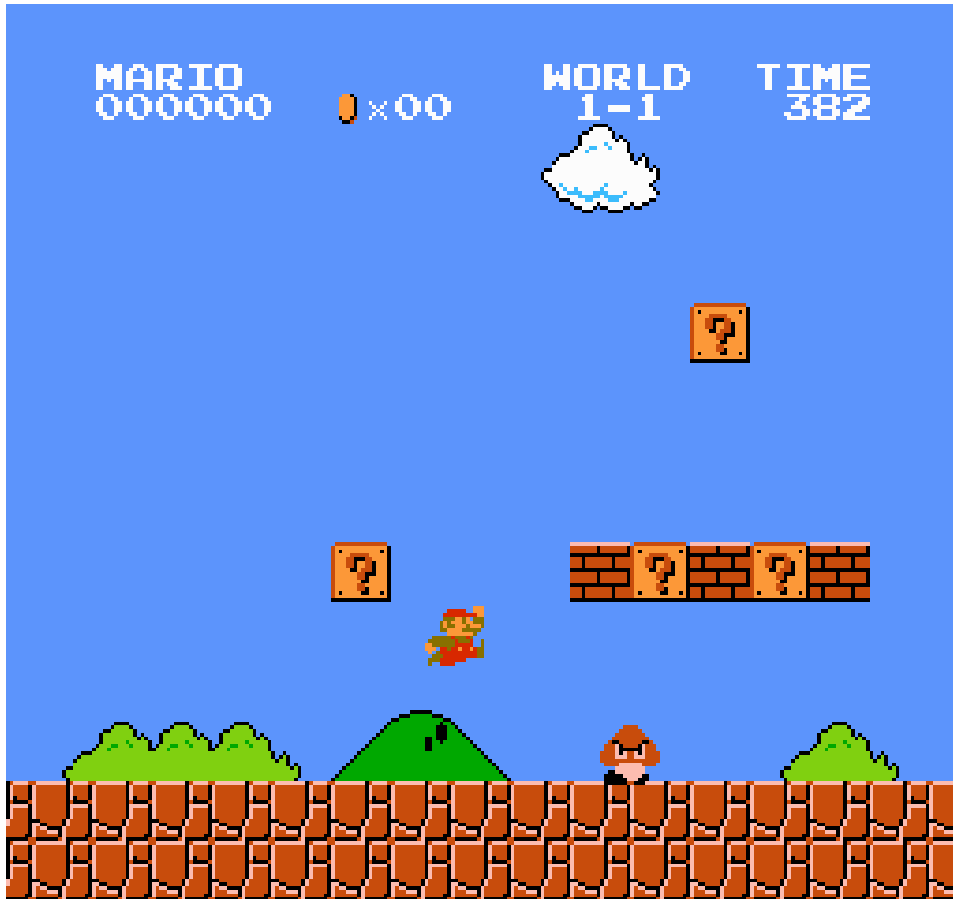
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$
    - $\dot{x} = [0 \ 0]$
  - If wall

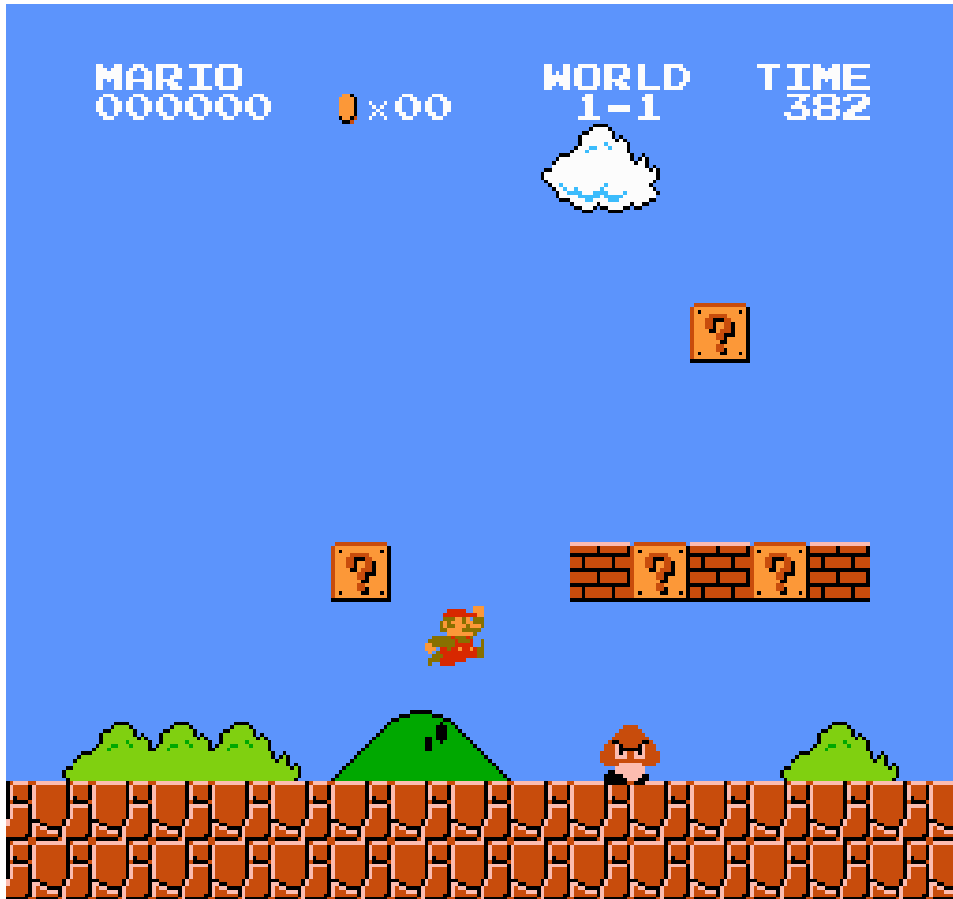
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$
    - $\dot{x} = [0 \ 0]$
  - If wall
    - $x = x$

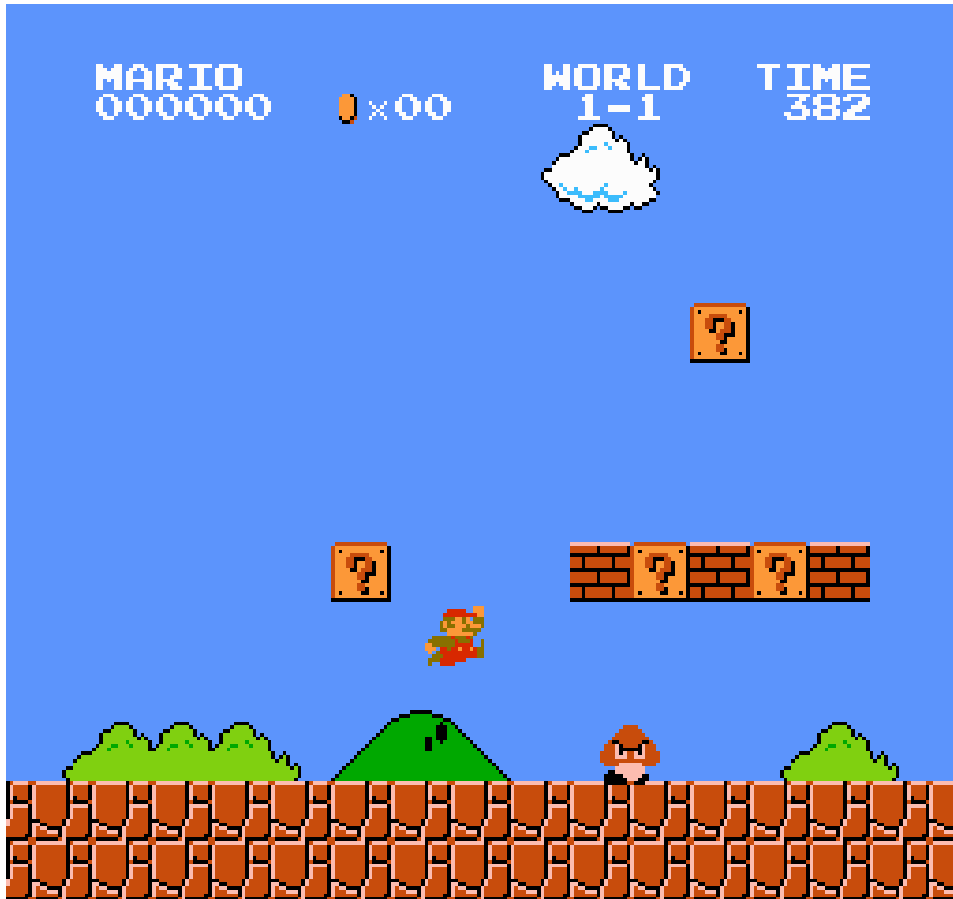
# Markov Decision Processes



## Transition Function ( $T$ )?

- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$
    - $\dot{x} = [0 \ 0]$
  - If wall
    - $x = x$
    - $\dot{x} = [0 \ 0]$

# Markov Decision Processes

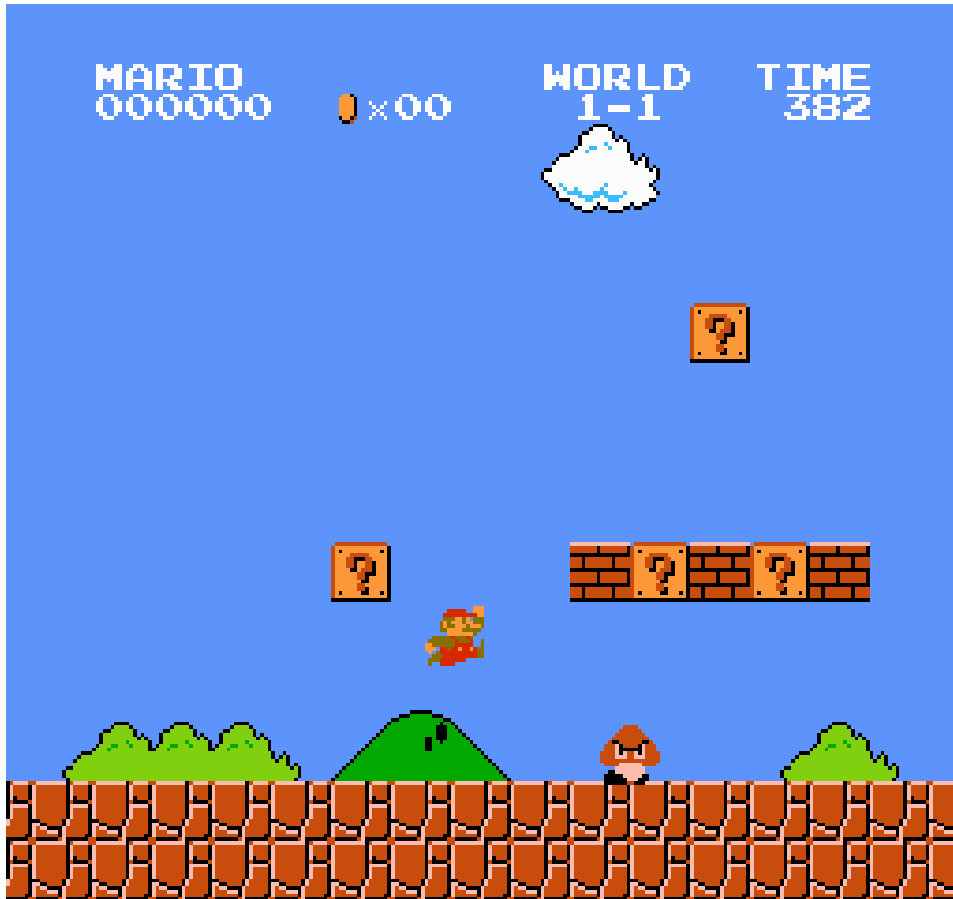


## Transition Function ( $T$ )?

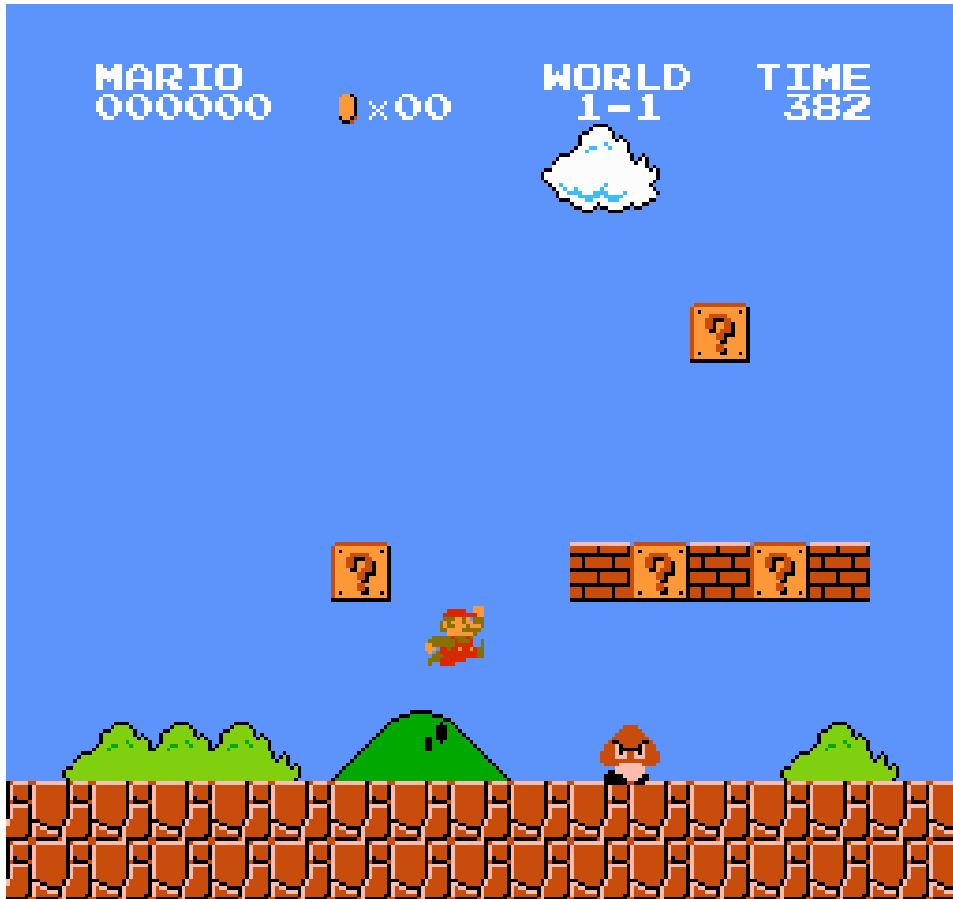
- $T([x \ \dot{x} \ ...], \rightarrow) = ?$ 
  - If enemy
    - $x = [0 \ 0]$
    - $\dot{x} = [0 \ 0]$
  - If wall
    - $x = x$
    - $\dot{x} = [0 \ 0]$
  - If ...

# Markov Decision Processes

- Reward ( $R$ )?

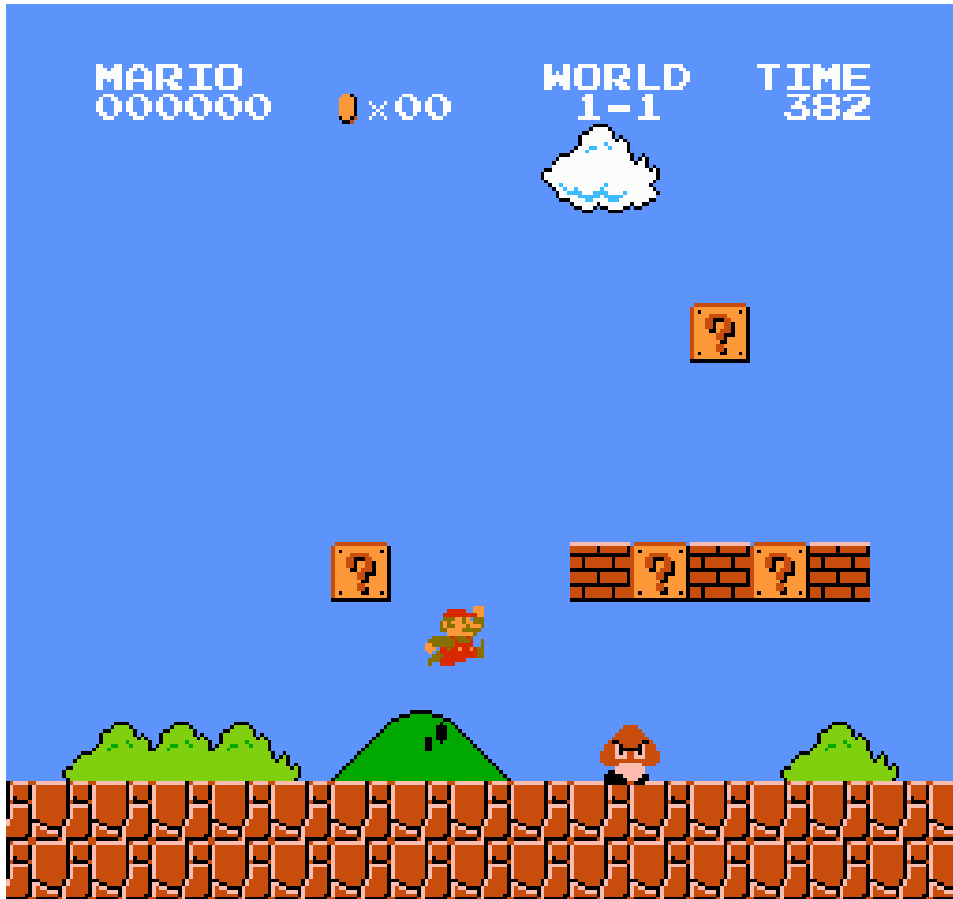


# Markov Decision Processes



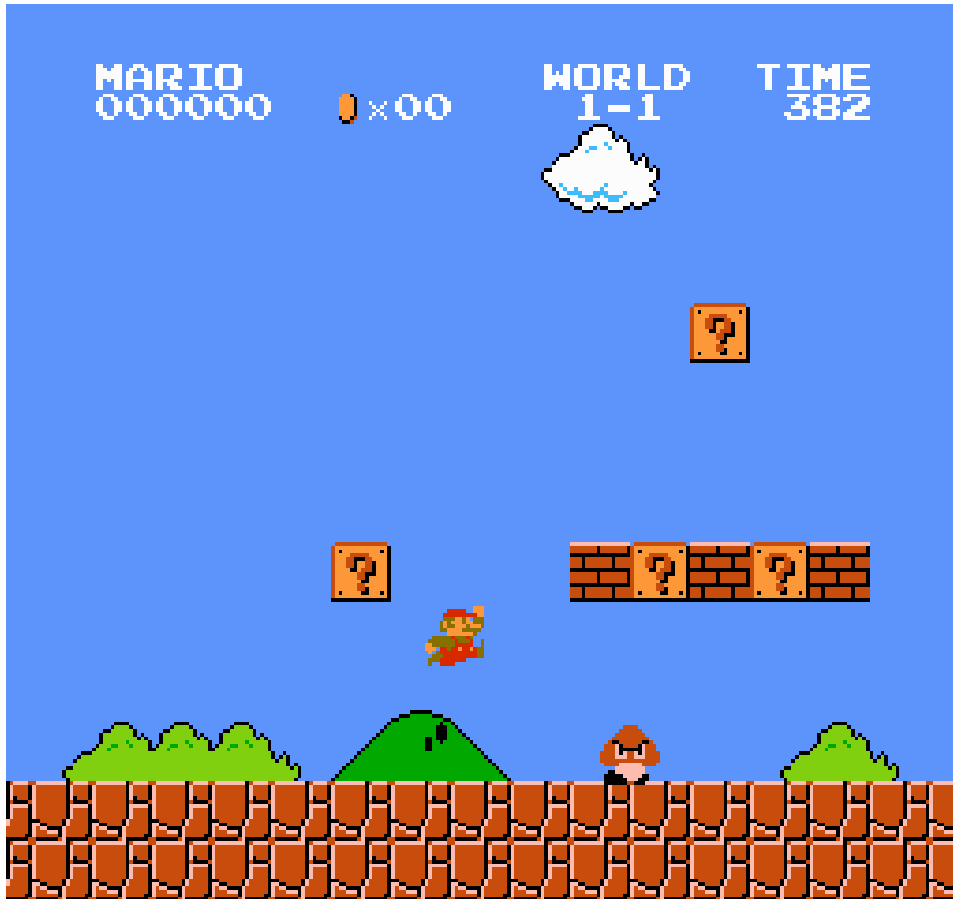
- Reward ( $R$ )?
  - 1 for beating the level and 0 otherwise

# Markov Decision Processes



- **Reward ( $R$ )?**
  - 1 for beating the level and 0 otherwise
  - Total score

# Markov Decision Processes



- **Reward ( $R$ )?**
  - 1 for beating the level and 0 otherwise
  - Total score
  - $1 \text{ for beating the level} + 0.01 \cdot \text{score}$

# Markov Decision Processes

- $S$ ✓
- $A$ ✓
- $T$ ✓
- $R$ ✓
- $\gamma$ ?

# Markov Decision Processes

Agent goal in RL is to maximize the **cumulative** reward

# Markov Decision Processes

Agent goal in RL is to maximize the **cumulative** reward

The cumulative reward is called the **return** ( $G$ )

$$G = \sum_{t=0}^{\infty} R(s_{t+1}) = \sum_{t=0}^{\infty} r_t$$

# Markov Decision Processes

Agent goal in RL is to maximize the **cumulative** reward

The cumulative reward is called the **return** ( $G$ )

$$G = \sum_{t=0}^{\infty} R(s_{t+1}) = \sum_{t=0}^{\infty} r_t$$

Note that we care about all future rewards, not just the current reward!

# Markov Decision Processes

Do humans maximize the return?

# Markov Decision Processes

Do humans maximize the return?

**Experiment:** one cheeseburger now, or two cheeseburgers tomorrow?

# Markov Decision Processes

Do humans maximize the return?

**Experiment:** one cheeseburger now, or two cheeseburgers tomorrow?

Humans and animals prefer rewards sooner.

# Markov Decision Processes

Do humans maximize the return?

**Experiment:** one cheeseburger now, or two cheeseburgers tomorrow?

Humans and animals prefer rewards sooner.

The **discount factor**  $\gamma$  injects this preference into RL. Results in the **discounted return** ( $G$ )

# Markov Decision Processes

Do humans maximize the return?

**Experiment:** one cheeseburger now, or two cheeseburgers tomorrow?

Humans and animals prefer rewards sooner.

The **discount factor**  $\gamma$  injects this preference into RL. Results in the **discounted return (G)**

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

$$0 \leq \gamma < 1$$

# Markov Decision Processes

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

$$0 \leq \gamma < 1$$

# Markov Decision Processes

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

$$0 \leq \gamma < 1$$

With a reward of 1 at each timestep and  $\gamma = 0.9$

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = 1 + 0.9 + 0.81 + \dots = \frac{1}{1 - \gamma} = 10$$

# Markov Decision Processes

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$$

$$0 \leq \gamma < 1$$

With a reward of 1 at each timestep and  $\gamma = 0.9$

$$G = \sum_{t=0}^{\infty} \gamma^t r_t = 1 + 0.9 + 0.81 + \dots = \frac{1}{1 - \gamma} = 10$$

We almost always choose to maximize the discounted return

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

**Environment:** University of Macau

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

**Environment:** University of Macau

**State:** My position, weather, and current activity

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

**Environment:** University of Macau

**State:** My position, weather, and current activity

**Action Space:** Buy a cheeseburger or go home

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

**Environment:** University of Macau

**State:** My position, weather, and current activity

**Action Space:** Buy a cheeseburger or go home

**Reward Function:**  $10 \cdot \text{eat}_{\text{burger}} - \text{rain}_{\text{mm/h}}$

# Markov Decision Processes

**Exercise:** Reinforcement learning also describes human and animal behaviors. How can you describe your behavior using reinforcement learning?

**Environment:** University of Macau

**State:** My position, weather, and current activity

**Action Space:** Buy a cheeseburger or go home

**Reward Function:**  $10 \cdot \text{eat}_{\text{burger}} - \text{rain}_{\text{mm/h}}$

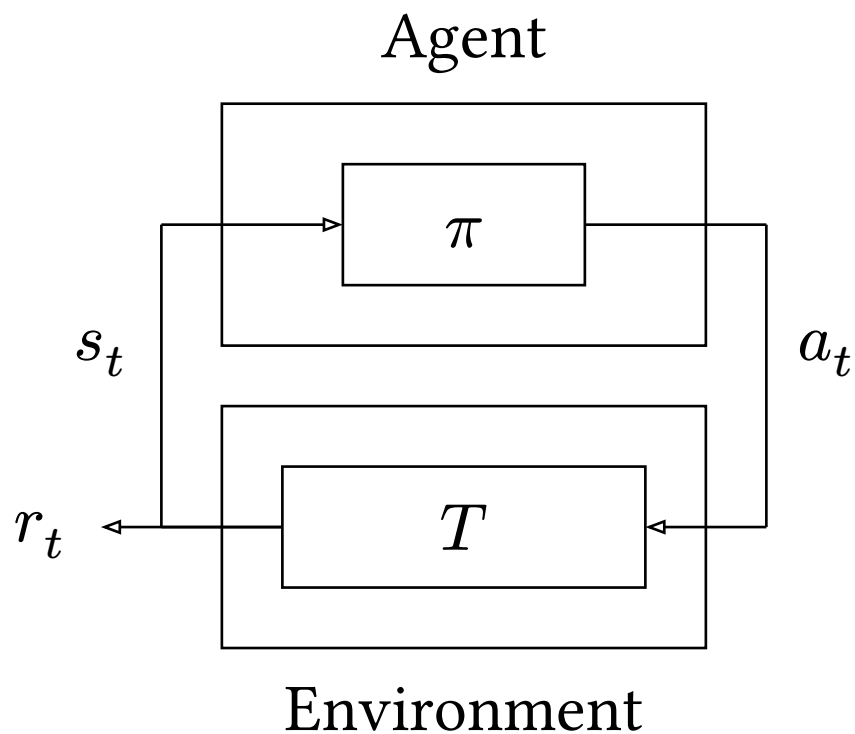
My brain implicitly makes this decision when I think about dinner

# Agent Policies

---

# Agent Policies

We have defined the environment,  
now define the agent

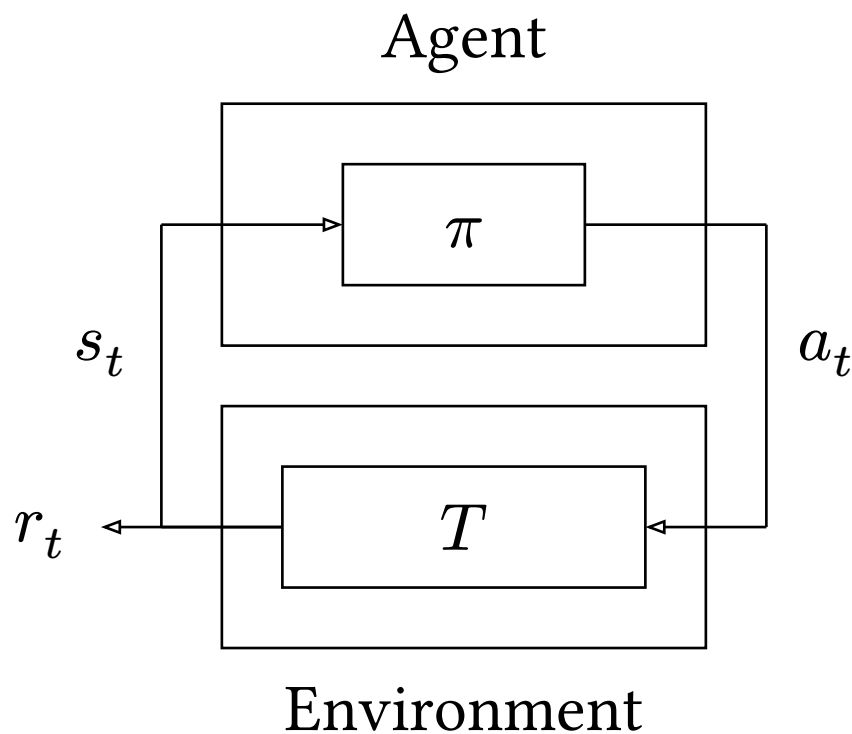


$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

# Agent Policies

We have defined the environment,  
now define the agent

Agent acts following a **policy**  $\pi$



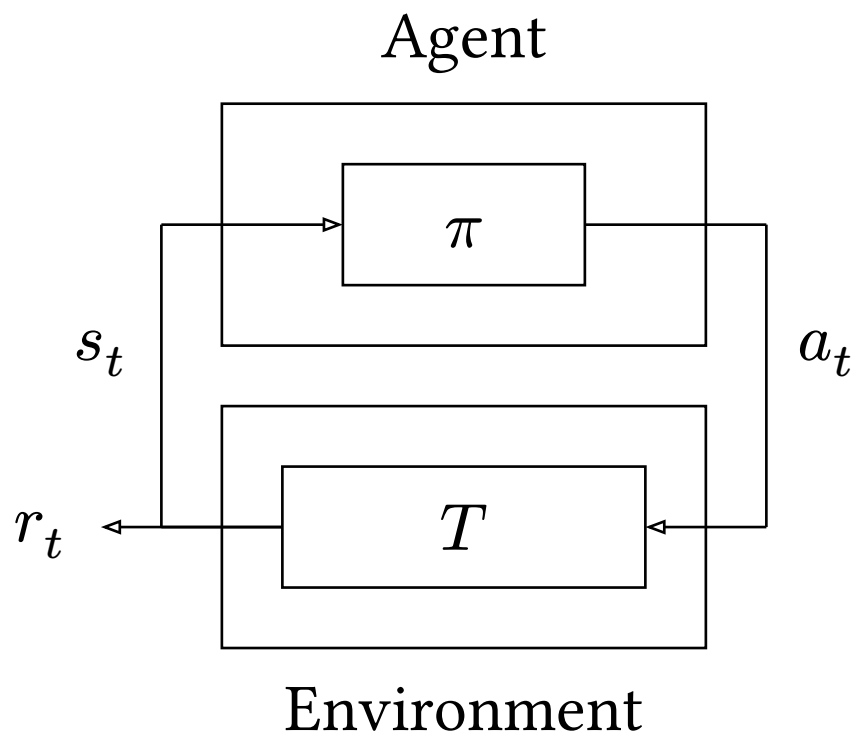
$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

# Agent Policies

We have defined the environment,  
now define the agent

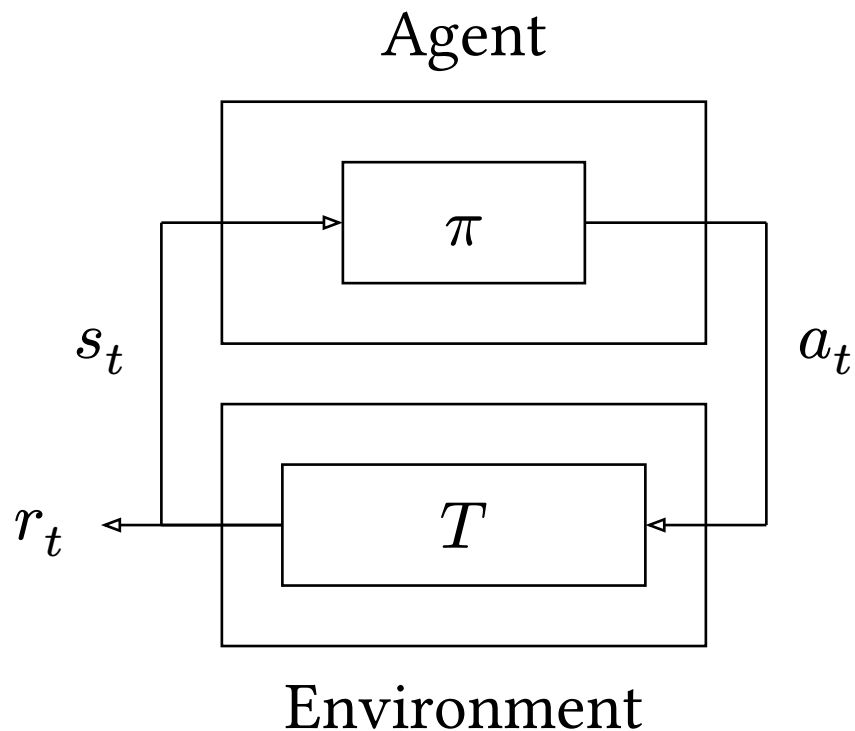
Agent acts following a **policy**  $\pi$

$$\pi : S \rightarrow \Delta A$$



$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

# Agent Policies



$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

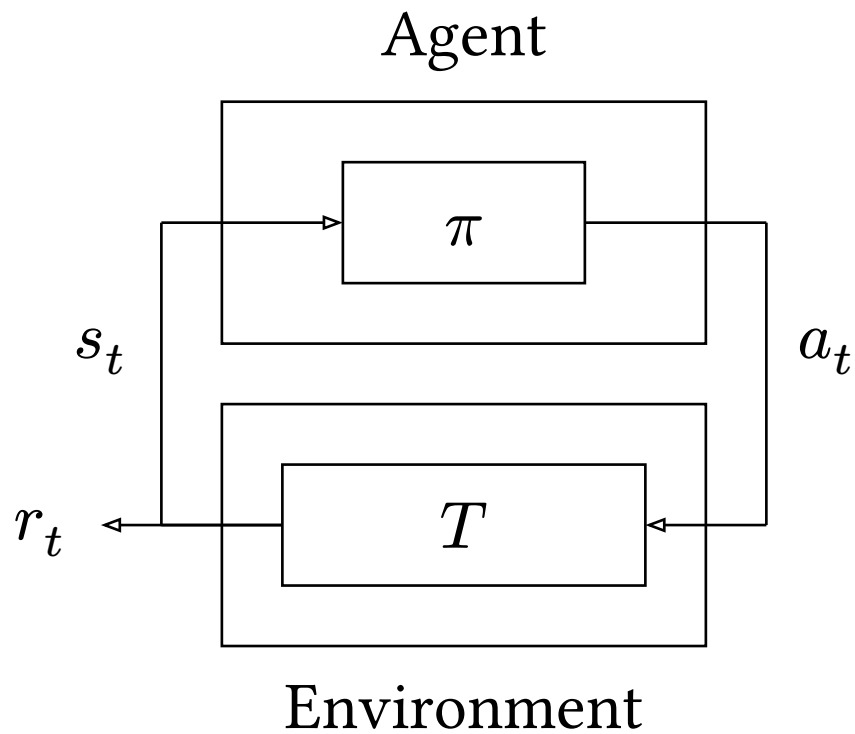
We have defined the environment,  
now define the agent

Agent acts following a **policy**  $\pi$

$$\pi : S \rightarrow \Delta A$$

Policy  $\pi$  maps state to action  
probabilities

# Agent Policies



$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

We have defined the environment,  
now define the agent

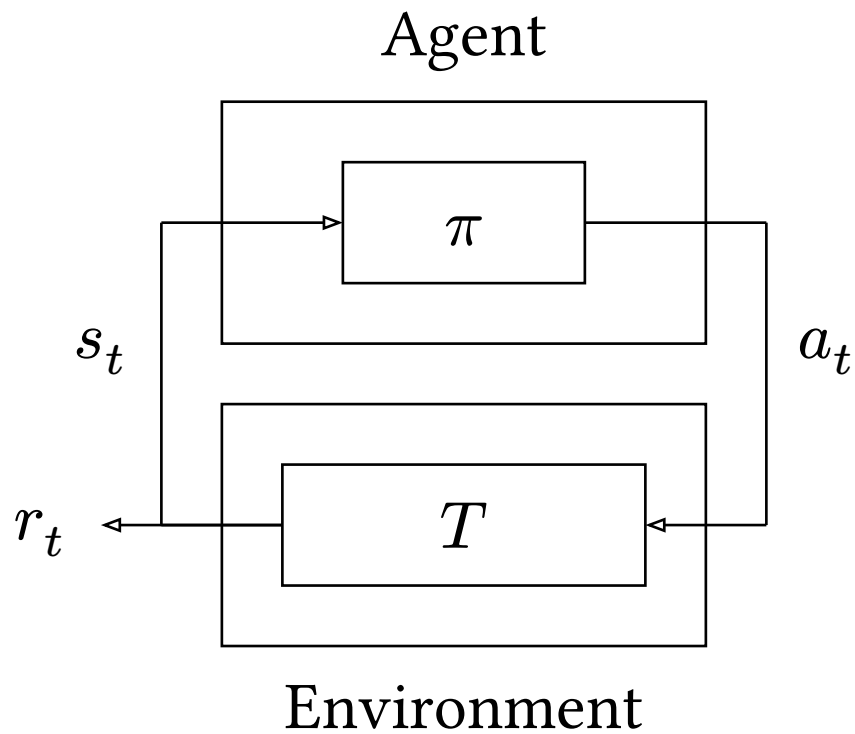
Agent acts following a **policy**  $\pi$

$$\pi : S \rightarrow \Delta A$$

Policy  $\pi$  maps state to action  
probabilities

$$a_t \sim \pi(\cdot | s_t)$$

# Agent Policies



$s_t$ : state,  $a_t$ : action,  
 $r_t$ : reward,  $\pi$ : policy,  
 $T$ : transition fn

We have defined the environment,  
now define the agent

Agent acts following a **policy**  $\pi$

$$\pi : S \rightarrow \Delta A$$

Policy  $\pi$  maps state to action  
probabilities

$$a_t \sim \pi(\cdot | s_t)$$

Policy fully determines agent  
behavior

# Agent Policies

$$a_t \sim \pi(\cdot | s_t)$$

# Agent Policies

$$a_t \sim \pi(\cdot | s_t)$$

Some policies are good and some are bad

# Agent Policies

$$a_t \sim \pi(\cdot | s_t)$$

Some policies are good and some are bad

**Question:** How do we determine how good a policy is?

# Agent Policies

$$a_t \sim \pi(\cdot | s_t)$$

Some policies are good and some are bad

**Question:** How do we determine how good a policy is?

**Hint:** What is our objective in RL?

# Agent Policies

$$a_t \sim \pi(\cdot | s_t)$$

Some policies are good and some are bad

**Question:** How do we determine how good a policy is?

**Hint:** What is our objective in RL? **Answer:** The return!

$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

# Agent Policies

$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

# Agent Policies

$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

The **expected return** tells us how good a specific policy  $\pi$  is

$$G_{\pi} = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

# Agent Policies

$$G = \sum_{t=0}^{\infty} \gamma^t r_t$$

The **expected return** tells us how good a specific policy  $\pi$  is

$$G_{\pi} = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

Tells us what return we should expect **on average** from following  $\pi$

# Policy Gradient

---

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

But it does not tell us how to find a good policy

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

But it does not tell us how to find a good policy

**Question:** How can we use  $G_\pi$  to find the best policy?

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

But it does not tell us how to find a good policy

**Question:** How can we use  $G_\pi$  to find the best policy?

**Answer:** Optimization problem, find  $\pi$  that maximizes  $G_\pi$

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

But it does not tell us how to find a good policy

**Question:** How can we use  $G_\pi$  to find the best policy?

**Answer:** Optimization problem, find  $\pi$  that maximizes  $G_\pi$

$$\pi_* = \arg \max_{\pi} G_\pi$$

# Policy Gradient

$G_\pi$  determines how good policy  $\pi$  is

$$G_\pi = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi(\cdot \mid s_t) \right]$$

But it does not tell us how to find a good policy

**Question:** How can we use  $G_\pi$  to find the best policy?

**Answer:** Optimization problem, find  $\pi$  that maximizes  $G_\pi$

$$\pi_* = \arg \max_{\pi} G_\pi$$

We call  $\pi_*$  the **optimal policy**

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

Today, let us find  $\pi_*$  using a deep learning algorithm

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

Today, let us find  $\pi_*$  using a deep learning algorithm

First, let us turn  $\pi$  into a neural network with parameters  $\theta$

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

Today, let us find  $\pi_*$  using a deep learning algorithm

First, let us turn  $\pi$  into a neural network with parameters  $\theta$

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

Today, let us find  $\pi_*$  using a deep learning algorithm

First, let us turn  $\pi$  into a neural network with parameters  $\theta$

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

$$\pi_* = \arg \max_{\pi} G_{\pi}$$

We use **algorithms** to search for  $\pi_*$

Today, let us find  $\pi_*$  using a deep learning algorithm

First, let us turn  $\pi$  into a neural network with parameters  $\theta$

$$a_t \sim \pi_{\theta}(\cdot \mid s_t)$$

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}}$$

Optimal parameters  $\theta$  maximize the expected return

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

New

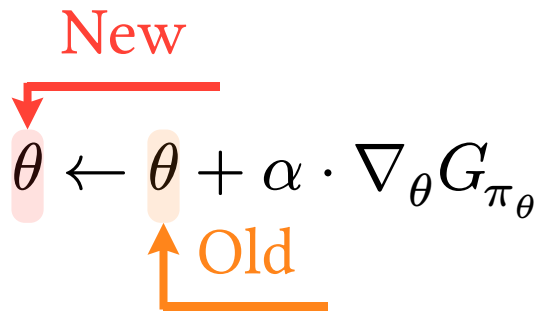

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

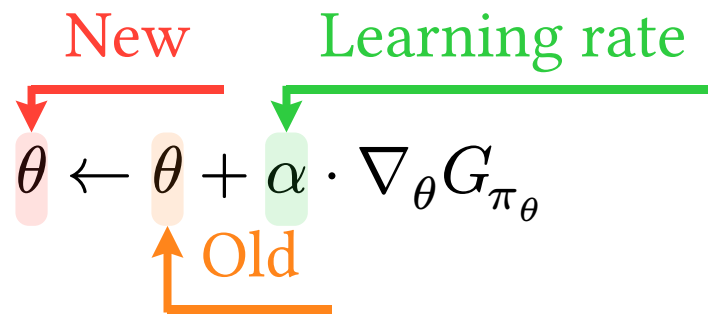

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!



The diagram illustrates the policy gradient update equation:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The equation is annotated with colored boxes and arrows: a red box around the left  $\theta$  is labeled "New" with a red arrow; an orange box around the right  $\theta$  is labeled "Old" with an orange arrow; a green box around  $\alpha$  is labeled "Learning rate" with a green arrow.

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

The diagram illustrates the policy gradient update rule:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The components are labeled as follows: 'New' (red) points to the updated parameter  $\theta$  on the left; 'Old' (orange) points to the current parameter  $\theta$  on the right; 'Learning rate' (green) points to the scalar  $\alpha$ ; and 'Gradient' (blue) points to the vector  $\nabla_{\theta} G_{\pi_{\theta}}$ .

# Policy Gradient

$$\theta_* = \arg \max_{\theta} G_{\pi_{\theta}} = \arg \max_{\theta} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \mid a_t \sim \pi_{\theta}(\cdot \mid s_t) \right]$$

**Question:** How do we find optimal parameters in deep learning?

**Answer:** Gradient ascent (descent)!

The diagram illustrates the policy gradient update rule:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The components are annotated as follows:

- New** (red arrow): Points to the updated parameter  $\theta$  on the left.
- Old** (orange arrow): Points to the current parameter  $\theta$  on the right.
- Learning rate** (green arrow): Points to the scalar  $\alpha$ .
- Gradient** (blue arrow): Points to the gradient term  $\nabla_{\theta} G_{\pi_{\theta}}$ .

# Policy Gradient

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

The diagram illustrates the Policy Gradient update equation:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The components are labeled with colored arrows: a red arrow points to the new  $\theta$  (labeled "New"), an orange arrow points to the old  $\theta$  (labeled "Old"), a green arrow points to the learning rate  $\alpha$  (labeled "learning rate"), and a blue arrow points to the gradient term  $\nabla_{\theta} G_{\pi_{\theta}}$  (labeled "Gradient").

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

# Policy Gradient

The diagram illustrates the update rule for the policy parameters  $\theta$  in a Policy Gradient method. The equation is  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . Annotations include: a red arrow pointing to the new  $\theta$  labeled "New"; an orange arrow pointing to the old  $\theta$  labeled "Old"; a green arrow pointing to the learning rate  $\alpha$  labeled "learning rate"; and a blue arrow pointing to the gradient term  $\nabla_{\theta} G_{\pi_{\theta}}$  labeled "Gradient".

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$$

**Question:** Can we compute  $\nabla_{\theta} G_{\pi_{\theta}}$ ?

# Policy Gradient

The diagram illustrates the policy gradient update equation:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The components are labeled with colored arrows: a red arrow labeled "New" points to the updated parameter  $\theta$ ; an orange arrow labeled "Old" points to the current parameter  $\theta$ ; a green arrow labeled "learning rate" points to the step size  $\alpha$ ; and a blue arrow labeled "Gradient" points to the gradient term  $\nabla_{\theta} G_{\pi_{\theta}}$ .

**Question:** Can we compute  $\nabla_{\theta} G_{\pi_{\theta}}$ ?

**Answer:** Generally, no. Requires taking gradients through transition function  $T$  which may be unknown or not-differentiable.

# Policy Gradient

The diagram illustrates the policy gradient update equation:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . The components are labeled with colored arrows: a red arrow labeled "New" points to the updated parameter  $\theta$ ; an orange arrow labeled "Old" points to the current parameter  $\theta$ ; a green arrow labeled "learning rate" points to the step size  $\alpha$ ; and a blue arrow labeled "Gradient" points to the gradient term  $\nabla_{\theta} G_{\pi_{\theta}}$ .

**Question:** Can we compute  $\nabla_{\theta} G_{\pi_{\theta}}$ ?

**Answer:** Generally, no. Requires taking gradients through transition function  $T$  which may be unknown or not-differentiable.

But using the **policy gradient theorem**, we can rewrite this as

$$\nabla_{\theta} G_{\pi_{\theta}} = \mathbb{E} \left[ G_{\pi_{\theta}} \nabla_{\theta} \log \pi_{\theta}(a \mid s) \mid s \in S \right]$$

# Policy Gradient

The diagram illustrates the policy gradient update equation:  $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} G_{\pi_{\theta}}$ . Annotations include: a red arrow labeled "New" pointing to the new parameter  $\theta$ ; an orange arrow labeled "Old" pointing to the old parameter  $\theta$ ; a green arrow labeled "learning rate" pointing to  $\alpha$ ; and a blue arrow labeled "Gradient" pointing to  $\nabla_{\theta} G_{\pi_{\theta}}$ .

**Question:** Can we compute  $\nabla_{\theta} G_{\pi_{\theta}}$ ?

**Answer:** Generally, no. Requires taking gradients through transition function  $T$  which may be unknown or not-differentiable.

But using the **policy gradient theorem**, we can rewrite this as

$$\nabla_{\theta} G_{\pi_{\theta}} = \mathbb{E} \left[ G_{\pi_{\theta}} \nabla_{\theta} \log \pi_{\theta}(a \mid s) \mid s \in S \right]$$

Policy gradient  $\nabla_{\theta} \log \pi_{\theta}$  is easy to compute (neural network)

# Policy Gradient

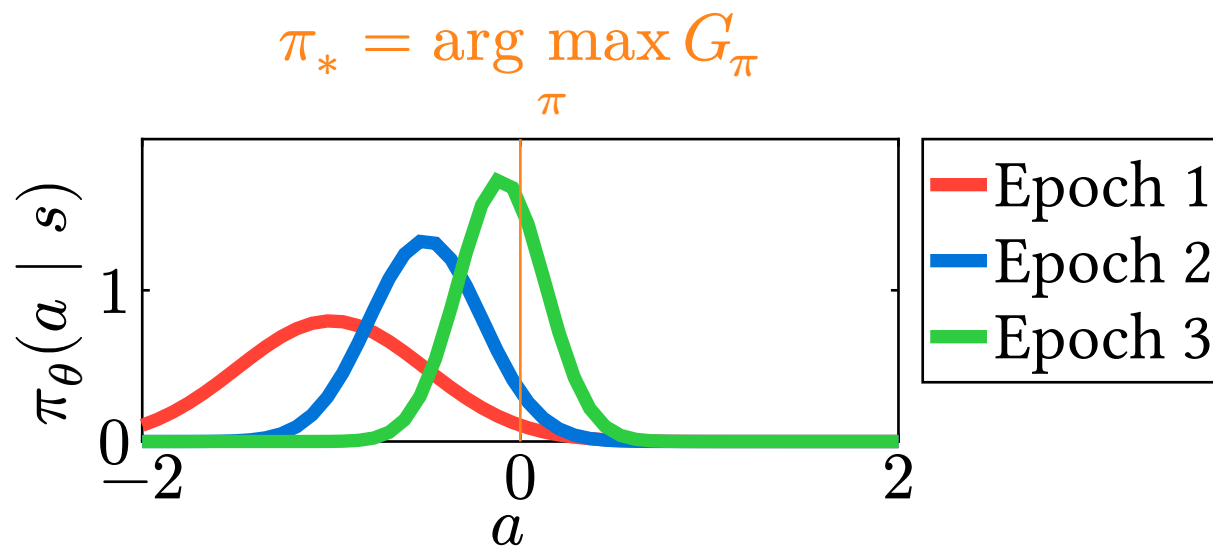
```
pi = MLP(state_size, action_size, final_act=softmax)
for epoch in range(num_epochs):
    # Run pi in the environment and collect a dataset of size n
    states, actions, rewards = run_pi(pi, environment)
    losses = []
    # Compute loss for each timestep i in the episode
    for i in range(len(rewards)):
        # Foreach i, compute return:  $\sum_{t=0}^n \gamma^t * r_t$ 
        G = sum(gamma**t * r for t, r in enumerate(rewards[i:]))
        # Compute probabilities for actions in dataset
        action_prob = pi(states[i])[actions[i]]
        # Torch uses gradient descent, loss is negative
        losses.append(-G * log(action_prob))
    sum(losses).backward(), opt.step(), opt.zero_grad()
```

# Policy Gradient

With this algorithm,  $\pi_\theta$  is **guaranteed** to converge to optimal policy  $\pi_*$

# Policy Gradient

With this algorithm,  $\pi_\theta$  is **guaranteed** to converge to optimal policy  $\pi_*$



Policy gradient can learn an optimal policy for any MDP. With enough data, this policy will perform better than any human.

# Policy Gradient

Better than any human:

# Policy Gradient

Better than any human:

- Starcraft II

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading
- Writing code

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading
- Writing code

Making progress, but not yet (need more training):

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading
- Writing code

Making progress, but not yet (need more training):

- Driving

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading
- Writing code

Making progress, but not yet (need more training):

- Driving
- Surgery

# Policy Gradient

Better than any human:

- Starcraft II
- Go (board game)
- Plasma confinement (nuclear fusion)
- Stock trading
- Writing code

Making progress, but not yet (need more training):

- Driving
- Surgery

Policy gradient powers all modern LLMs. With enough time/data, you can learn anything!

# Policy Gradient

Using two hours of data, we can learn some convincing robot policies!

- <https://youtu.be/gU1UEZVlUk0>
- <https://youtu.be/ecbwzohfyGM>